Client internals and version profiles, part 3 of 4

Client internals and version profiles, part 3 of 4

This part contains 37 notes, from `client-internals/foot-test.md` through `client-internals/offline-play.md`.

`client-internals/foot-test.md`

Foot test (movement collision)

Date: 2026-10-14

Superseded. This note records the single-point wall test recovered from the SDL reimplementation (2dadventureclient). The real clients (2.05 through 2.31) use a six-probe body-box routine with corner nudges and flush snaps. See Seihai `documentation/walking-collision.md` for the current description, and `archival/foot-test-sdl.md` for the preserved SDL snapshot.

The original content is retained below for reference.

Derived from the SDL reimplementation of the client movement loop and the type table. No decompile material.

Single-point test

Arrow keys set a pixel step of `16 * 0.30` and a sprite direction (0 up, 1 left, 2 down, 3 right). The wall test is a single point, not a box or a sweep:

```
testx = (pixelX + moveX + 32) / 16
testy = (pixelY + moveY + 44) / 16
```

If `isOnWall(testx, testy)` is false, the new position is accepted. If true, the step is rejected. There is no slide, no corner resolution, and no water slowdown in this loop. `isOnWater` exists but is not consulted by the keyboard path.

Predicates

- Outside the board on x or y: wall (edge is solid until a LINK says otherwise).
- Inside: type byte at `levelTiles[round(y)*64 + round(x)]` ≥ 20 is wall; type == 11 is water (does not block).
- Chair, bed, swamp, and lava types are stored and ignored by this path. Type 12 (old note: lava) is walkable here.

Stamp consequence

The 2×2 Bush stamp writes ids 2, 3, 18, 19. Those type bytes are 20/22, so the bush is wall. The stamp file itself carries no collision column; the foot test decides after the ids are written.

Official ~2.05 used a different body-block mask and type-0 allow lists. The measured path above keeps only the two predicates.

`client-internals/ledges-push-pull.md`

Ledge jumps, pushing and pulling (2.05)

Date: 2026-10-28. Read from the 2.05 rev 2 client; tile lists from the 2.22 install's `arrays.dat`. **[2.05]** unless tagged. No addresses, routine names or decompiler text. Units are tiles; a tick is the 50 ms game tick, and a walk step is one arrow key in one tick. Test level: `new3.nw` (Seihai `client/tests/fixtures/levels/new3.nw`).

Summary

1. Running into a wall shows the `push` gani once the player has been stopped for 18 steps in a row (0.9 s).
2. Running into a ledge jumps the player off after the same 18 steps, with `jump.wav`: 8 ticks along a fixed curve. No gani changes in the air (the walk gani stays); keys are not read.
3. Running into a stone block pushes it one tile on the 29th blocked step, then every 20 steps. The player follows one tile; `stonemove.wav`.
4. A on a wall (nothing to lift) grabs it (`grab` gani) and holds while A is held. The arrow away from the wall plays `pull`. Every 18 ticks the client tries a pull. A pullable block comes one tile and the player steps back (`lift.wav`, or `lift2.wav` by a player flag not traced). Anything else, including a wall or a rock too heavy for the glove, does not move: the pull gani keeps playing and the `pulled` trigger still fires every 18 ticks. Letting go of the arrow returns to holding; letting go of A ends the grab.
5. Every push and pull try fires a trigger action (`pushed` / `pulled`) at a point in front, which NPCs there see as `actionpushed` / `actionpulled`; online it also goes to the server (PLI_TRIGGERACTION, 38) and to UDP peers (id 26).

Player modes involved

mode	meaning
0	normal
1	pushing (last step refused)
2	ledge jump (8 ticks)
4	lifting (5 ticks, then carrying)
5	grab / pull

The blocked-step count and the grab's pull cycle share one per-player counter.

The ledge test

The client counts blocked steps. A step counts when it is refused and no corner nudge was applied; a free step or a nudge sets the count back to 0, and so does the first refused step after a free one.

- At count 18 the ledge test runs. At 38 the count goes back to 18, so it runs again every 20 steps.
- Test point, from the player's position, by facing (up, left, down, right): `(1.5, 0.5)`, `(0, 2)`, `(1.5, 3.5)`, `(3, 2)`. The tile there must be a ledge tile **for that facing**. In the usual tile mode those are the four lists of `arrays.dat` array 8 (2.05 calls the class tile type 21, "jumping stone"):

facing	ledge tiles (2.22 `arrays.dat`)
up	190, 191, 4045, 4046
left	212, 228, 563, 579, 4028, 4044
down	185, 186, 4061, 4062

facing	ledge tiles (2.22 `arrays.dat`)
right	273, 289, 4031, 4047

- Landing point, from `(x + 1.5, y + 2)`: `(0, -5)`, `(-6.5, 3)`, `(0, 7)`, `(6.5, 3)`. It must pass the ordinary walking test (not a wall, not a chest). If it does not, the player just keeps pushing.
- The jump starts from the position after the refused step. Per tick `i = 0..7` (the first in the same tick as the start):

facing	position
up	`y0 - [1.5, 2.7, 3.8, 4.5, 5.0, 5.2, 5.2, 5.0][i]`
down	`y0 + [-0.2, 0.2, 0.5, 1.5, 2.8, 4.0, 5.4, 7.0][i]`
left / right	`x0 -/+ [1, 2, 3, 3.8, 4.5, 5.2, 5.8, 6.5][i]`, `y0 + [-0.2, -0.5, 0, 0.5, 1.1, 1.7, 2.3, 3.0][i]`

- `sounds/jump.wav` plays at `(x + 1.5, y + 2)` when it starts. Keys are not read until it lands.

In `new3.nw` (decoded with the 2.22 lists): the up ledges are the cliff edge at y = 14 and 15 (x 18 to 46); the left ledges are the two slanted cliff sides, x 14 to 16 and x 48 to 56, between y = 27 and 50. There are no down or right ledges. The push block is at 34,19, the pull block at 31,24, and 42,26 to 43,28 holds a mix of push and pull pieces that neither moves cleanly.

Pushing

At count 28 (checked before the step is counted, so on the 29th blocked step), and every 20 after, the client tries a push:

1. It fires the trigger action `pushed` at the test point above, with the facing as its parameter. NPCs there get `actionpushed`. Online the 2.05 client also sends it to the server and to its peers.
2. It reads the tile under that point. If it is one of the four tiles of the pushable block (`arrays.dat` array 16, entry 0: 1764, 1765 / 1780, 1781), that tile's place in the block gives the block's top-left. Only that one tile is compared, as in 2.05.
3. The two cells the block moves into must not be a wall, a chest, water, a chair or the lower half of a bed. From the block's top-left: up `(0,-1) (1,-1)`, left `(-1,0) (-1,1)`, down `(0,2) (1,2)`, right `(2,0) (2,1)`.
4. The block moves one tile. Each of its old cells first takes the tile two cells ahead, then the block is written one cell ahead. So the row the block leaves gets the floor that was in front of it.
5. The 2.05 client sends the 4 x 4 area around the old spot as a board modify, moves the player one tile in the facing, and plays `sounds/stonemove.wav`.

Pushed and pulled NPCs

When no block is found, the same routine looks for an NPC at the probe point.

- **Lookup.** The level's NPC hit test at the point. A character NPC is hit in `x + 0.5 .. x + 2.5`, `y + 1 .. y + 3`; a `setshape2` NPC by its shape's tile types; a GANI NPC by its animation; any other by its image box `[x, x + w/16) x [y, y + h/16)`, where a transparent pixel is a miss for plain image NPCs. The NPC needs `canbepushed` (GS1 command 44, cleared by 45). **[2.05]**
- **Edge.** The NPC is left out of its own test while every point along the edge it moves into is checked, one per tile from its left or top, `t` while `t < x + w` (or `y + h`): up `(t, y - 1)`, left `(x - 1, t)`, down `(t, y + h)`, right `(x + w, t)`. Sizes are in pixels / 16; character and GANI NPCs count as 48 x 48. Each

point goes through the client's blocked-point test: other players (online only), blocking NPCs, two level object lists (most likely dropped objects and chests), and wall tiles. Water does not stop an NPC. **[2.05]**
- **Move.** The NPC moves exactly one tile, the player one tile the same way, then `waspushed` fires on the NPC. `actionpushed` has already fired. The routine reports success, so the same sound as for a block plays. A character NPC that crosses a water edge gets the splash and switches between `swim` and `idle`. **[2.05]**
- **Server.** Nothing is sent for the NPC move: the NPC's x and y setters only store the value. The `pushed` trigger action is the only message. **[2.05]**
- 2.22 is the same, but its NPC lookup also searches the neighbouring map parts on a gmap. **[2.22]**

The trigger action on the wire

The `pushed` / `pulled` try goes to the server as `PLI_TRIGGERACTION` (38): GINT 0 (three spaces, no NPC), `GCHAR x*2`, `GCHAR y*2`, then the action name, and `,` plus the parameter when there is one, the direction as a decimal string (`pushed,0`). 2.05 also sends it to UDP peers (id 26). **[2.05]**

Grabbing and pulling

- **Order on A**: throw what is carried; lift; grab.
- **Grab test** at `(1.5, 0.9)`, `(0, 2)`, `(1.5, 3.1)`, `(3, 2)` by facing: a wall tile there, or an NPC that blocks. Nothing is grabbed when another player stands there.
- The grab lasts while A is held. Arrow keys do not move the player or turn them.
- **The pull cycle.** Holding the arrow away from the wall sets a counter to 2 and starts the pull pose. Each tick the counter goes 2, 3, ... 19, then back to 2 (18 ticks). Whenever the arrow is held at 19 the client tries a pull. Letting go of any arrow puts the counter back to 1 (just holding).
- **A pull try** fires `pulled` one tile out from the body (`(1.5, 0)`, `(-0.5, 2)`, `(1.5, 4)`, `(3.5, 2)`), with the opposite of the facing as its parameter, then reads the tile at the grab point. If it is a tile of the pullable block (array 16, entry 1: 1998, 1999 / 2014, 2015), the two cells behind the player must be free (from the block's top-left: up `(0,4) (1,4)`, left `(4,0) (4,1)`, down `(0,-3) (1,-3)`, right `(-3,0) (-3,1)`). NPCs with `canbepulled` are moved the same way and get `waspulled`. Then the block moves one tile towards the player (each old cell first takes the tile two cells behind), the player steps back one tile, and `lift.wav` plays (or `lift2.wav`, by a player flag that was not traced).
- **Pulled NPCs.** With no pullable block at the grab point, an NPC there with `canbepulled` (command 46, cleared by 47) is pulled the same way as a pushed one. First the two cells behind the player must be free, from the whole-tile grab cell: up `(0,3) (1,3)`, left `(3,0) (3,1)`, down `(0,-3) (1,-3)`, right `(-3,0) (-3,1)`. Then the edge facing the player is checked: up `(t, y + h)`, left `(x + w, t)`, down `(t, y - 1)`, right `(x - 1, t)`. The NPC and the player move one tile back and `waspulled` fires. Nothing goes to the server for the NPC. **[2.05]**
- **The futile pull.** On anything else (a wall, a rock too heavy for the glove, a push block, the mixed block pieces at 42,26 in `new3.nw`) nothing moves. The `pulled` trigger still fires every 18 ticks, and the pull pose keeps playing.

Sprites on the wire

From the animation step table (`decomp-findings-2026-10-24/sprite-step-table.tsv`): `push` is steps 14 + frame (S 11, 12, 13, 11, 12), `grab` step 19 (S 14), `pull` steps 20 + frame (S 15, 16, 17). The jump sends no sprite of its own.

arrays.dat

- **Array 8** holds four lists, one per facing (up, left, down, right): the ledge tiles. This is what tile type 21 ("jumping stone") means in the usual tile mode: the test asks whether the tile is in the list for the player's current facing. In the alternative tile mode (a tiledefs sheet of type 1 or 2) type 21 is instead the tiles in rows 30 and 31, columns 8 to 15, of the 16-column sheet.
- **Array 16** holds two 2 x 2 blocks, `[column][row]`: entry 0 the pushable block (1764, 1780 / 1765, 1781), entry 1 the pullable block (1998, 2014 / 1999, 2015). **[2.22]**

2.22 and 2.31

Both behave as 2.05 for everything above: the same counts, probe points, flight curve, block tiles, triggers and pull cycle (the number tables are identical in all three). The differences:

- **Grab on NPCs.** 2.05 grabs a blocking NPC only when a per-NPC flag is clear (most likely the character-shape flag). 2.22 and 2.31 grab any NPC that blocks there, and the lookup also checks the neighbouring map parts. The player and wall checks are unchanged. **[2.22] [2.31]**
- **Stop gates.** Walking, the ledge test, the A key and other gameplay code do nothing while either of two server-set flags is on. Server packets 176 and 177 set them; most likely these are the full-stop packets. **[2.22] [2.31]**
- **Sounds and custom ganis.** `jump.wav`, `stonemove.wav`, the pull sound and footsteps play only while the current gani is one of the built-in names (idle, walk, ..., push, grab, pull). The jump, push and pull still happen with a custom gani; it is expected to bring its own sounds. The sound names are also given bare (`jump.wav`) instead of under `sounds/`. **[2.22] [2.31]**
- **No jump gani** in either version: the flight sets none, so the walk or push gani stays. **[2.22] [2.31]**
- A further flag on the server connection also stops gani changes, footsteps and the A key; what it means was not found. **[2.22]**

Open

- ~~What is drawn right after A is let go while `pull` is playing.~~ Seen live (2026-10-30, `player-mannerisms-live.md`): `pull` keeps looping until the next key moves the player.
- Releasing A or the arrow mid-pull in 2.22 / 2.31 was not checked step by step.

`client-internals/carry-hide-and-glove.md`

Hiding under a carried object, and the glove (2.05)

Date: 2026-10-29. Read from the 2.05 rev 2 client; tile lists from the 2.22 install's `arrays.dat`. **[2.05]** unless tagged. No addresses or decompiler text. Key slots are graalconfig's: D is the item key, S attack, A action.

Hiding (mode 14)

- **D while carrying** a bush, a light stone, a black stone or an NPC hides the player under it. The carry sprites that allow it are 261 (bush), 263 (light stone) and 239 (black stone). A pot (265) or a sign (267): D does nothing.
- **D again**, or **A**, comes out: the player goes back to carrying (`carrystill`). Out from under a bush (not an NPC) the client starts break effect 0 (the leaves) at `(x + 0.5, y + 1)`.
- **S** throws, from hiding too.

- **Poses.** `hiddenstill` standing, `hidden` walking. The walk routine applies a factor of 0.2 while hidden, the speed: 0.1 tile per tick, seen live. **[live]**
- **Hits.** The hit gate refuses hits on a hidden player unless a player byte that was not traced is set.
- **Wire.** `hiddenstill` is animation step 24, sprite 18 (as `carrystill`); `hidden` is steps 25 to 28, sprites 19, 20, 19, 18. STATUS gets bit 0x02. The carry sprite prop does not change.
- Seen live (2026-10-30, `player-mannerisms-live.md`): stopping while hidden shows `hiddenstill`, and walking hidden is 0.1 tile per tick (the 0.2 factor is the speed).

Glove power and the lift table

- The lift routine reads the tile at the A key's facing point and tries the first **glove + 2** entries of `arrays.dat` array 11 (lift tiles; array 12 holds what each leaves behind). All four tiles must match.
- Array 11 in the 2.22 file: bush 2/3/18/19, sign 512/513/528/529, **pot** 684/685/700/701, light stone 34/35/50/51, black stone 990/991/1006/1007. The carry sprites in the same order are 261, 267, 265, 263, 239.
- Every player starts at **glove 1** (bush, sign, pot). The `glove1` item raises the power to at least 2 (adds the light stone), `glove2` to at least 3 (the black stone). GServer agrees: its default account has `GLOVEP 1` and its glove items set 2 and 3. **[srv]**
- Online the power is player prop 6 (GLOVEPOWER).
- An NPC is lifted only when its script set `canbecarried` (command 41, cleared by 42), at the same point and with no glove check. It leaves the level's NPC list and is carried. No packet goes out for it.

`client-internals/water-and-ganis.md`

Water, splashes and the built-in animation names

Date: 2026-10-28. Read from the 2.05 rev 2 client and checked against its executable's string table. **[2.05]** These notes contain no addresses, routine names or decompiler text.

Built-in animations

The client keeps one table of animation names, which it plays as `<name>.gani`. In table order:

#	name	#	name	#	name
0	idle	11	carrystill	22	hurt
1	walk	12	hiddenstill	23	ridehurt
2	walkslow	13	carry	24	dead
3	kick	14	hidden	25	maps1
4	sword	15	shoot	26	maps2
5	ridesword	16	ridestill	27	maps3
6	spin	17	ridefire	28	swim
7	push	18	rideeat	29	lava
8	grab	19	ride	30	carried
9	pull	20	sit		
10	lift	21	sleep		

Uses that were seen in the code:

- The sword swing plays `sword`; on a horse, `ridesword`.
- The spin (a second S press during a swing, with the has-spin status) plays `spin`.

- The A key's grab plays `grab`, and its lift plays `lift`.
- The item key's "lay item" mode plays `hiddenstill`.
- Firing a weapon on a horse plays `ridefire`.
- A throw sets the player back to `idle`.
- Swimming plays `swim`, or `lava` on a lava tile (type 12).

Water

The client checks the player's tile on every game tick. It skips the check while the sword swings, while riding, and when the player is not under local control.

- **Into a liquid tile.**
  - When not swimming already, the client starts a splash, which is break effect 5, at the player's position.
  - When the player was not walking at the time (a warp or a knock), it starts two more splashes.
  - It then enters swim mode with `swim`, or `lava` when the tile type is 12.
- **Off a liquid tile while swimming.** One splash, then back to standing.
- **Liquid test.** The lava animation is chosen inside the water branch, so the client's liquid test is true for lava tiles too. **[inf]** The test itself is a jump table that was not followed.

Break effect sounds

Each break effect plays a sound as it starts, at `(x + 1, y + 1)` from its anchor:

- `sounds/water.wav` for the splash (effect 5);
- `sounds/crush.wav` for every other effect.

`client-internals/feature-specs/11-emotes-ganis.md`

11 Emotes and non-walk GANIs

Snapshots read: Graal_analysis `8da6800` (`formats/gani.txt`, `protocol/udp-p2p.md`, `packet-layouts.md`, `versions-133-139.md`, `player-props.md`), Seihai `06aacfd` (`assets/gani.rs`), GServer-v2 (`msgPLI_SHOWIMG`, `TPlayerProps.cpp`).

What it is

Two separate things.

A. Animation state

- **2.1+ (incl. 2.22, 2.31)**: `PLPROP_GANI` (10) is a string gani name (`idle`, `walk`, `sword`, `spin`, ...). The receiver fetches `<name>.gani` with WANTFILE if it lacks it. GServer stores it and forwards it; when it is `spin` it fabricates four HITOBJECTS (02).
- **2.05**: prop 10 is the pre-2.1 bow form. The animation is in `PLPROP_SPRITE` (17): `sprite % 4` = direction and `sprite / 4` = a sprite number S. **[2.05][dec]** The 2.05 decompile (2026-10-24) gives the exact table, see "2.05 sprite prop" below. The UDP movement datagram has a trailing byte that looked like the first letter of the animation name (`W`, `I`, `S`); the client also keeps a table that maps gani-name substrings to one letter (`attack` A, `carry` C, `dead` D, `hurt` H, `idle` I, `kick` K, `lift` L, `ride` R, `sit` T, `spin` U, `swim` V,

`sword` S, `walk` W, `def` X and a few more, used to shorten cached gani file names; full table in `../decomp-findings-2026-10-24/gani-short-codes.tsv`).
- Props that interact: CARRYSPRITE (19), STATUS bits, HORSEGIF (21), HEADGIF/BODYIMG, `PLPROP_GATTRIB1-30` (gani parameters `#P1..`).
- GANI file grammar (SPRITE, ANI/ANIEND frames, PLAYSOUND, LOOP, SETBACKTO, DEFAULTHEAD...) is in `formats/gani.txt`. Seihai's parser covers frames, loops and `PLAYSOUND`; effects and scripts are not run.

B. Emotes

- Emotes are **images shown over a player's head**. On the wire: `PLI_SHOWIMG` (24) client to server, rest passed through; GServer re-emits it as `PLO_SHOWIMG` (32) with `s player id` in front to the level area (`msgPLI_SHOWIMG`, `sendPacketToLevelArea`). In 2.05 the same payload also goes over UDP id 15. **[srv] [cap]**
- Layout seen: four GCHARs (index, x, y, name length) then the image name, for example the AFK emoticon `emoticon_afk_stay.mng`, re-sent about every 3 s while shown. Trailing bytes follow. **[cap]**
- Triggers: 1.38+ Ctrl+letter (A AFK, B BRB, Z Zzz stay until the player moves; others from D, E, F, G, H, L, M, O, P, R, S, T, W, X; 1.39 adds C, K, Y). The assets include an `emoticons` folder (29 png/mng files). 2.17+ also has chat-line emoticons.
- **Correction (2026-10-24 decompile of 2.05 rev2; the 1.39 build has the same routine).** The key path is **local only and sends no packet**. Ctrl+A to Ctrl+Z (no text field focused, player not dead or locked) lowercases the letter and walks the client's loaded file list for the **first file whose name starts with `emoticon_<letter>`** and whose extension is png, mng or gif (a prefix match: Ctrl+A matches `emoticon_afk_stay.mng` if it is the first `emoticon_a...` file in the list). The image is drawn at the player's x+2, y-5 (a little lower when a second flag is set) for **50 ticks (2.5 s)**. There is no letter-to-name table and no hard-coded AFK/BRB/Zzz string in the client: the mapping is whatever files exist. `PLI_SHOWIMG` (24) is sent **only by the script command `showimg`**, never by the key handler. The earlier wording of this spec (Ctrl+letter sends SHOWIMG) was an inference from the captured packets and is wrong for the key path.
- **Open:** how other players see a Ctrl+letter emote. The captured SHOWIMG frames (AFK image re-sent about every 3 s) are not explained by the key handler; they may come from a server-side NPC or script that calls `showimg`, or from a path not yet traced. Treat the emote *send* as unknown. (Lab batch c, 2.31 and 2.22: the stock server relays any client 24 payload unchanged as 32 and no client reacted to a 1-byte payload; the real 2.31 payload is a single byte 9 sent about every 5.1 s, not an emote form: `../decomp-findings-2026-10-24-b/08-lab-findings-levelchange-24-178.md`.)
- `PLO_SHOWIMGNPC` (166) shows images on NPCs.

2.05 sprite prop [2.05][dec]

Derived from the 2.05 rev2 decompile; tables are in `../decomp-findings-2026-10-24/` (`sprite-step-table.tsv`, `gani-to-step.tsv`, `gani-name-table.tsv`, `status-bits-prop18.tsv`, `player-modes.tsv`). - Byte on the wire = 32 + direction + 4 x S (the 32 is the usual GCHAR offset, so the decoded prop is `dir + 4 x S`, matching `sprite % 4` and `sprite / 4` above). - S comes from the current animation step through a step table, and **only while the active gani is the default `def` gani and the step is 38 or lower**. Other ganis do not change the sprite number. The sender writes 0 for names it does not know. A hurt player is special-cased: S = 32 (decoded `0x80 + dir`). - Step to S (steps 0 to 38): `0, 4, 5, 4, 3, 2, 1, 2, 3, 6, 7, 8, 9, 10, 11, 12, 13, 11, 12, 14, 15, 16, 17, 23, 18, 19, 20, 19, 18, 21, 22, 21, 18, 24, 25, 26, 27, 28, 29`. Steps 9 to 13 (S 6 to 10) are the sword swing and the only ones that can hit (see 02). - Gani name to step rule: idle and def = frame; walk and walkslow = frame+1; kick 9; sword, ridesword, spin = 9 + (frame mod 5);

push 14+frame; grab 19; pull 20+frame; lift 23; carrystill and hiddenstill 24; carry and hidden 25+frame; shoot 33; ridestill 34, ridefire 35, rideeat 36; ride 35 + (frame and 1); sit 37; sleep 38; hurt and ridehurt 39; dead 40 (not sent). - **Swim, lava and dead are not on the wire.** All three are sent as sprite 0 (idle) plus direction. Swim is a local mode (10) entered on water tile type 11 (with a splash); lava (tile type 12) shows the local `lava` gani. The dead state is carried only by the STATUS prop dead bit (see 04). There is **no swim sprite**, so a 2.05 client cannot tell a swimmer from an idle player by prop 17. - The 31 built-in gani names (index 0 to 30): idle, walk, walkslow, kick, sword, ridesword, spin, push, grab, pull, lift, carrystill, hiddenstill, carry, hidden, shoot, ridestill, ridefire, rideeat, ride, sit, sleep, hurt, ridehurt, dead, maps1, maps2, maps3, swim, lava, carried. - Caveat: the "only for `def`" condition was read from the sender and is recorded as observed, not confirmed on the wire. - STATUS prop (18) bits: 0x01 editor or paused, 0x02 hidden (mode 14 or hidden timer), 0x04 male, 0x08 dead (see 04), 0x10 weapons allowed, 0x20 sword hidden (the `hidesword` timer, `../status-bit-0x20.md`), 0x40 has charged spin.

Seihai today

- Parses gani files and keeps a walk animation player (`set_walk_gani`, `walk_frame`). Does not send SPRITE animation indexes beyond walking, does not draw other players' non-walk animation, and has no emote send/receive (PLO 32 not handled). Other players are drawn from props.

Steps for Claude

1. For 2.22+ profiles: on movement/attack/lift state changes send `PLPROP_GANI` with the gani name; draw other players using the gani named in their props, loading it via the asset path or WANTFILE.
2. For 2.05: send prop 17 as `dir + 4 x S` using the step table above (idle = S 0, walk = frame+1 mapped through the table, sword swing S 6 to 10, and so on). Send sprite 0 plus direction while swimming and while dead (no swim sprite exists). To draw other 2.05 players, invert the table: `S -> pose`, with the caveat that several steps share an S. Do not invent a swim sprite.
3. Implement `PLO_SHOWIMG` (32): decode, spawn a timed overlay above that player (x,y given in the packet, name from the packet), expire after about 3 s unless refreshed.
4. Emote key (2.05 behaviour): on Ctrl+letter, find the first file in the loaded asset list whose name starts with `emoticon_<lowercase letter>` and has a png, mng or gif extension; draw it locally at the player's x+2, y-5 for 50 ticks (2.5 s). **Send nothing.** Send `PLI_SHOWIMG` (24) only from a script `showimg` command, with the captured field layout. Whether other players should see a key emote is open; if Seihai wants that, treat it as a Seihai choice (sending SHOWIMG is protocol-legal) and mark it so.
5. Draw `.mng` as a first frame only (GIF/MNG decoders are not in Seihai); skip PNG variants gracefully.
6. Add the `hurt`, `dead`, `sword`, `lift`, `grab`, `push` poses as the combat/inventory/death work (02-04) needs them.

How to test

- Lab: Wine 2.05 client sends an AFK emoticon (Ctrl+A) next to a Seihai session; check PLO 32 arrives and decodes. Replay the existing two-player capture through `SessionState` for ids 15 and 24.
- Load every `.gani` in the lab world `ganis` folder with `Gani::parse` as a smoke test.

NEEDS

Filled by the 2026-10-24 2.05 decompile: the sprite-index to animation table and the Ctrl+letter lookup (both above). 2.22+ needs nothing: names are plain strings.

Still open: - How other players see an emote (the key path sends nothing; see the correction above). A lab capture of a real 2.05 client pressing Ctrl+A next to a second client would settle it. - Swim on the wire: none found in 2.05. Whether 2.22 sends a `swim` gani name (prop 10) is covered by the 2.1+ notes, not by this decompile. - Some mode values (see `../decomp-findings-2026-10-24/player-modes.tsv`). - 1.39 sprite and gani behaviour was not re-read (the emote routine is the same).

`client-internals/feature-specs/06-gmap-overworld.md`

06 GMAP overworld and map links

Snapshots read: Graal_analysis `8da6800` (`formats/gmap.md`), Seihai `06aacfd`, GServer-v2 (`TMap.cpp`, `TPlayer.cpp` `setLevel`/`sendLevel`/`msgPLI_ADJACENTLEVEL`, `TPlayerProps.cpp`, `TLevelLink.cpp`, `TPlayerLogin.cpp`). Folded here from the public-server report: **PLO_BIGMAP 171, PLO_MINIMAP 172, PLO_SETACTIVELEVEL 156, PLO_LEVELMODTIME 39**.

Three different "maps" (do not mix)

1. **Level links** (`PLO_LEVELLINK` 1): `dest x y w h newx newy` text per link. Walking into the rectangle makes the client send `PLI_LEVELWARP` (0) / `PLI_LEVELWARPMOD` (30). `newx`/`newy` may be `playerx`/`playery`. `TLevelLink::parseLinkStr` accepts level names with spaces. Seihai parses PLO 1 and has `send_levelwarp`.
2. **bigmap-style server maps** (`maps =` option, `bigmap.txt` text grid, `MapType::BIGMAP`): server only. It lets the server know which levels are adjacent so players are visible and can chat across edges. The client does nothing special. Level-to-cell data comes from `TMap::loadMapLevels`.
3. **GMAP** (`gmaps =` option, `GRMAP001` file, `MapType::GMAP`): the client walks continuously across level borders. Separate again: `bigmap =` / `minimap =` options (4 comma fields `text,image,x,y`) make the server send `PLO_BIGMAP` (171) and `PLO_MINIMAP` (172) at login (2.1+, in `sendLoginClient`). These are overlay settings for the client's map screen and minimap: file names to fetch plus a default x,y for heads of players off the map. Harmless to skip. The notes warn that setting `bigmap` breaks gmaps.

GMAP wire behaviour [srv]

- File keys (`formats/gmap.md`): WIDTH, HEIGHT, LEVELNAMES block (quoted CSV rows), MAPIMG, MINIMAPIMG, NOAUTOMAPPING, LOADFULLMAP, LOADATSTART.
- Entering a gmap level (2.1+ clients): `PLO_PLAYERWARP2` (49): `c x*2, c y*2, c z+50, c gmapX, c gmapY` + gmap file name. 2.05 has no handler for 49 and gets plain `PLAYERWARP` (14).
- In `sendLevel`: after the level's board, links, chests, horses, baddies, the server sends `PLO_LEVELNAME` (6) with the **gmap name** (not the level) and, when this is the first level, `PLO_SETACTIVELEVEL` (156) with the gmap name before the NPC props, then `PLO_GHOSTICON` 0.
- Neighbours: the client sends `PLI_ADJACENTLEVEL` (35) `i5 modtime + level`; the server replies with that level's data (`sendLevel(..., fromAdjacent)`), then restores `LEVELNAME` to the gmap. The 3x3 neighbourhood is preloaded this way.
- Crossing a border is **client-driven**: the client updates `PLPROP_GMAPLEVELX` (43) / `GMAPLEVELY` (44) and X/Y in gmap coordinates. For each of those props the server looks up the new level name in the map (`TMap::getLevelAt`), leaves the old level and enters the new one with an unknown mod time.
- Props: `PLPROP_CURLEVEL` carries the gmap name while on a gmap; X/Y inside a gmap level are level-local plus the cell; newer clients (2.30+) also send X2/Y2/Z2 pixel props (78-80).
- UDP id 24 ("entered level") goes to peers in the 3x3 neighbourhood (see 07).

- `PLO_LEVELMODTIME` (39): `i5 modtime` after each full level; keep it per level so that the next entry can use `LEVELWARPMOD` (30) or `ADJACENTLEVEL` with a cached board.

Lab capture of a real 2.22 client crossing gmap edges, 2026-10-24 [cap]

Setup: a 2x2 gmap of open-grass levels; the client walked east, south, west and north across all four edges. - **Login.** The server sends `PLO_PLAYERWARP2` (49) with x*2*, y*2*, z+50, gmap cell x, gmap cell y, then the **gmap name** (not a level name). Then `PLO_LEVELNAME` (6) with the level, a second `6` with the gmap file name, 174, 10, and `156 SETACTIVELEVEL` = the gmap name. The `.gmap` file arrives as a file transfer (rawdata + file) because the client asks for it. The first big server frame also carries the neighbour levels' boards. The client sends `PLI_ADJACENTLEVEL` (35, `i5 modtime` + name) at login for the three neighbour levels (modtime 0 on a cold cache). - **A crossing uses only normal props; there is no level-warp packet.** - The client sets the flags `gr.x` / `gr.y` (`PLI 18`, flag set) on every half-tile step. They carry the **global** position: local + 64 * cell, so they run past 63.5. (Earlier notes in `version-profiles/2.22-2.31.md` say these flag sets come from the server's movement NPC rather than from client code; both fit if the server-sent movement weapon runs inside the client and the client sends the resulting flag sets. The capture cannot tell which.) - East: in the frame where global x reaches 64.0 the client sends `PLI 39` (the tampercheck constant, see 08), `gr.x=64`, and a props message with local X = 0 and `GMAPLEVELX` (prop 43) = 1. - South: the frame at global y 64.0 carries `PLI 39` and the flag only; the props message with local Y = 0.5 and `GMAPLEVELY` (prop 44) = 1 follows in the next half-step frame, because position props go out only on whole-tile steps. - West: `PLI 39` goes out at global x 63.5; the next props message has X = 63.0 and prop 43 = 0. North is the same with Y = 63.0 and prop 44 = 0. - Rule: local = global - 64 * cell; the cell prop changes in the same props message as the wrapped coordinate; position props use whole-tile steps; **2.22 re-sends `PLI 39` on each crossing**. - **Server answer to each crossing** (one frame): `6` new level name, `39` (server to client level mod time, only when the client does not have the level cached), `0` (an empty board packet), `6` gmap name, `174`, `10`, `156` gmap name. A second visit to a level omits the server's `39`. - The merged Seihai, connected to this lab, treated the gmap name in `PLO_PLAYERWARP2` as a level name and requested `<gmap>.nw`, which does not exist. Gmap support (this spec) is still not implemented; the above is the data it needs.

Seihai today

- Profile flag `levels.gmap` exists (set per version profile in `profile.rs`; check which ones before relying on it); the `LEVELNAMES` text is not parsed. PLO 49, 171, 172, 156, 39, 6 (LEVELNAME) is handled; PLI 35 is never sent. The rest are not implemented. 156/39/171/172 are counted as unhandled in the public-server report.
- `draw_minimap` draws a 64x64 level minimap (not the gmap).
- Single-level warps (`PLAYERWARP` 14, `send_levelwarp`) work.

Steps for Claude

1. Add `formats::gmap` parser: `GRMAP001`, WIDTH, HEIGHT, LEVELNAMES rows (quoted CSV, lower-cased lookup), MAPIMG/MINIMAPIMG, flags. Test with a hand-written 2x2 file.
2. Handle PLO 49 (decode the layout above) to enter a gmap; store `gmap_name`, `cell_x`, `cell_y`. Handle PLO 6 on a gmap as "the map name". Store 156 and 39 (modtime per level).
3. Fetch the `.gmap` with WANTFILE if absent, then the levels of the 3x3 neighbourhood: send `ADJACENTLEVEL` (35) for each neighbour.
4. Render neighbour levels offset by 64 tiles; compose collision across seams. Position in world = cell*64 + local.
5. On crossing an edge, send `PLPROP_GMAPLEVELX/Y` (43/44) plus the wrapped X/Y in one prop message. Let the server answer with the new level data; keep the old level data for one tick to avoid flashing.

6. Parse 171/172 into settings (file names + default x,y); fetch images only if a map screen exists. Otherwise ignore.
7. Gate everything on `profile.levels.gmap` so 2.05 keeps working.

How to test

- Lab: copy a 2x2 set of lab levels, write `lab2x2.gmap`, set `gmaps = lab2x2.gmap` in a **copy** of the lab `serveroptions.txt`, restart the lab gserver, log in with a 2.22 client (Seihai `--version 2.22`), warp into one of the levels and walk across an edge. Confirm PLO 49, 6, 156 and the neighbour level packets in the decoded frames. Display :9 only.
- Unit tests for the parser and for the cell/world coordinate maths.

NEEDS

~~a lab capture of a real 2.22 client crossing a gmap edge~~ Done 2026-10-24 (see above; 2.22 has no X2/Y2 props). ~~a 2.31 crossing~~ Captured in the lab 2026-10-24 (batch b2, seven level entries): 2.31 sends `PLI_ENTERLEVEL` (46, `GCHAR 1`) first, then the props message with the cell prop (43 or 44) and the wrapped local coordinate, and does not re-send `PLI 39` at the crossing; the server's answer matches 2.22 (`../decomp-findings-2026-10-24-b/07-lab-findings-46-186-200.md`). Still open, thin evidence: the Z2 prop at a crossing, and how the 2.17+ client lays out the gmap overview on the map screen.

`client-internals/feature-specs/02-weapons-combat.md`

02 Weapons and combat

Snapshots read: Graal_analysis `8da6800`, Seihai `06aacfd`, GServer-v2 (local checkout).

Weapon lists (server to client) [srv][2.05]

PLO	Name	Layout
33	NPCWEAPONADD	`str name` + NPC prop stream (image, script). A name the player already has is ignored. 2.05 fires `playerenters` on it.
34	NPCWEAPONDEL	rest = name
43	DEFAULTWEAPON	`c item index` (0-24, see 03). A built-in weapon: bow, bomb, sword, etc.
197	(bytecode)	v4+ only; replaces the script for GS2. 2.05 never sees it.

Client to server: `PLI_WEAPONADD` (33) type 0 = `c item index`; type 1 = `i3 npc id`. 2.05 only sends type 0. `PLI_NPCWEAPONDEL` (29) = name.

GServer builds the list at login (`sendLogin`): clear (`PLO_CLEARWEAPONS`), the account's weapons, protected weapons, then default weapons if the `defaultweapons` option is on. The weapon script text comes from `TWeapon`; for pre-v4 clients the script is sent as plain GS1 text.

Combat packets [srv][2.05][cap]

The client is authoritative. It sends its own action and the server re-emits it to the rest of the level with `s player id` in front. In 2.05 most of these also go over UDP (see 07); the TCP copy is sent only when some player in the level has no UDP port, or an NPC server exists.

PLI	PLO	Name	Client layout	Notes
4	11	BOMBADD	`c x*2, c y*2, c power(+8 = mine), c fuse`	fuse seen: 55 (about 2.75 s)
5	12	BOMBDEL	`c x*2, c y*2`	PLO has no player id
9	19	ARROWADD	`c x*2, c y*2, c flags, c type`, last byte offset 33	flags: dir&3, +4 effect, +8 owned
10	20	FIRESPY	`c (a*8+b)`	
11	21	THROWCARRIED	empty	
26	40	HURTPLAYER	`s victim, c (dx+4)*16, c (dy+4)*16, c power` [+ `i3 npc`]	receiver: power = (v-32)/2 hearts, cap 1.0
27	36	EXPLOSION	`c radius, c x*2, c y*2, c power(+8 mine)`	power & 7 clamped 1-3
36	46	HITOBJECTS	`c power*2, c x*2, c y*2` [+ `i3 npc`]	sword-swing "hit" broadcast
16	27	BADDYHURT	`c id, c dx, c dy, c power`	applied only by the level leader
15	2	BADDYPROPS	`c id` + baddy prop stream	server forwards to everyone except the leader
17	-	BADDYADD	`c x*2, c y*2, c type, c power` + image	server creates the baddy and answers with props
38	48	TRIGGERACTION	`i3 npc, c x*2, c y*2` + action text	half-tiles in 2.05

Server behaviour (`TPlayer.cpp`): - `msgPLI_HURTPLAYER`: victim lookup; if the victim has the paused status bit, nothing happens; otherwise forwards to the victim with the attacker id. - `msgPLI_BADDYHURT`: sent only to the level leader (first player in the level's list), who owns baddy AI. `msgPLI_BADDYPROPS` stores the props and forwards to non-leaders. `msgPLI_BADDYADD` caps power at 12 half-hearts and marks it non-respawning. - `TLevelBaddy::setProps`: mode die starts a 2 s timer to dead; mode dead starts the `baddyrespawntime` timer (60 s lab default) or removes the baddy; with `baddyitems` it drops an item on death. - Level objects (`TLevel::doTimedEvents`) respawn cut bushes after `respawntime` (15 s) by re-sending BOARDMODIFY. - `msgPLI_CLAIMPKER` (14): the dead player names the killer; the server adjusts kills/AP (and a rating update in sparring zones). - `PLO_ISLEADER` (10): "you run this level's baddies". Seihai counts it as unhandled. It matters only if Seihai wants to simulate baddies. - Spin attack (2.1+ only, `PLPROP_GANI` = `spin`, handled in `TPlayerProps.cpp`): the server itself broadcasts four `HITOBJECTS` packets around the player (4 half-tiles above, below, left, right of the centre) so other clients register the spin. 2.05 has its own sender for HITOBJECTS (36); the exact trigger in a normal swing is not traced. How 2.05 receives hits is in `protocol/packet-layouts.md` "Hits": HURTPLAYER applies only if the attacker is within 10 tiles and the local player is hurtable; HITOBJECTS only if the player's centre is within 2 tiles of (x, y).

2.05 sword swing and hit box [2.05][dec]

From the 2.05 rev2 decompile (2026-10-24); details in `../decomp-findings-2026-10-24/04-sword-hitbox.md`. - **Start of a swing.** Needs weapons allowed by the level or server, sword power not 0, not swimming without a horse, and not already in the charged-spin mode. A normal swing is mode 6. A spin is mode 13: it needs the has-spin flag (STATUS bit 0x40) and starts when S is pressed again while mode 6 is still running. It is not a hold-to-charge; see `../player-mannerisms-live.md`. The client plays the sword, ridesword or spin gani and a sound, then checks for a cuttable bush at the sword tip. The sprite sent to others follows animation steps 9 to 13 (S 6 to 10, see 11). - **The victim's client decides a hit**; the attacker only broadcasts position, direction and sprite. A hit counts when **all** of these hold: - the attacker is on step 9 to 13 (S 6 to 10); - the victim is **0 to 4 tiles ahead** of the attacker in the facing direction; - the sideways offset is **under 2 tiles**; - the victim's **AP is not exactly 100** (such players cannot be hit); - the victim is hurtable (not in mode 3, mode 10, dead or editor/pause). - **Damage** is inferred to be sword power divided by 2 hearts. Not confirmed from a capture; treat it as a Seihai default. - **Explosions:** a bomb hit needs the victim within 1 tile on both axes of the blast, with knockback of 0.5 or 1.0 tile. - **After a hit** the victim goes to hurt mode 3 for 40 ticks (cannot be hurt again), and if hearts reach 0 or below the victim client sends `PLI_CLAIMPKER` (14) naming the attacker (see 04). - Open: how this relates to the `HURTPLAYER` (26) packet in 2.05 (the decompile findings do not say whether the attacker also sends it for a sword hit); frame timing in milliseconds (comes from the gani files, not the client); 1.39 sword and hit box (not read).

Seihai today

- `SessionState` keeps `weapons` (names and scripts) and `default_weapons`; it does **not** show or select them, send any combat packet, or decode 11/12/19/20/21/36/40/46/48 (they count as unhandled).
- Offline mode has a sword model in the web-Seihai design notes (`formats/seihai-systems.md`). Those numbers are Seihai's own, not 2.05 values.
- No HURTPLAYER handling: Seihai never loses hearts from another player.

Steps for Claude

1. Parse server packets 11, 12, 19, 20, 21, 36, 40, 46, 48 into events on `SessionState` (list of timed objects: bombs, arrows, hits). Decode per the table. Ignore silently if the level is not the named level.
2. Render bombs and arrows with the existing sprite sheet; remove on BOMBDEL, on explosion or timeout.
3. HURTPLAYER receive: subtract power from `CURPOWER` locally, apply knockback `(v-0x60)/16`, start an invulnerability window, then send the new `PLPROP_CURPOWER` (the server accepts hearts from the client, and refuses to raise them for AP under 40). 3a. Seihai as a **victim** should evaluate sword hits itself with the 2.05 rule above (reach 0 to 4 tiles ahead, lateral under 2, attacker on S 6 to 10, own AP not 100, not already hurt or swimming), and send CLAIMPKER when hearts reach zero.
4. Send side, in this order: BOMBADD (4) with the owner flag, explosion (27) when the fuse ends, BOARDMODIFY (1) for any tile the blast changes, HITOBJECTS (36) when your sword or bomb effect lands (trigger not traced, see NEEDS), HURTPLAYER (26) when your sword box hits another player in the level.
5. Weapons UI: list `weapons` and default weapons; pick with the item key; for GS1 weapons run the script through the existing `scripting` module.
6. Baddies: parse BADDYPROPS (2); only simulate if `ISLEADER` (10) has been received. First pass: draw them from props only.
7. Track an invalid-packet budget: GServer disconnects a client after the 6th unknown id (`msgPLI_NULL`), so never send ids absent from the profile's `pli` set.

How to test

- Two Seihai or Seihai plus one Wine 2.05 client in the lab level, `bushitems = true`. Drop a bomb next to bushes: expect BOMBADD, then eight BOARDMODIFY runs, ITEMADD drops, then a respawn BOARDMODIFY 15 s later (matches the existing capture in `packet-layouts.md`).
- Replay the existing decoded two-player captures through `SessionState` (examples `lab_replay`) and assert the new event lists.
- GServer rebroadcast can be checked with a second raw client from `examples/session_probe.rs`.

NEEDS

Nothing for the protocol. The 2.05 hit-box geometry and the frames that can hit are now known (above). Still open, for feel only: when exactly 2.05 sends `HITOBJECTS` (36) in a normal swing, the real damage formula (inferred power / 2), and the 1.39 sword. The server does not check any of it, so any plausible box interoperates.

`client-internals/combat-200-204.md`

Combat in 2.00 to 2.04 (victim's client)

How the 2.00, 2.01, 2.02, 2.03 and 2.04 clients judge a sword hit and the hurt packet, compared with the 2.05 rev 2 rules in `decomp-findings-2026-10-24-d/` and `damage-sources.md`. Read in the binaries of 2.00 (rev 0), 2.01, 2.02, 2.03 and 2.04 (MD5s in `build-catalogue.md`), each run through the same Ghidra pipeline as `build-diff-204-205.md` (11.4.2, Delphi compiler spec). The 2.00 revs 1 to 3 and the 2.02 revs 1 and 2 were not read.

Cleaned note: no addresses, no decompiler output, no generated names. Nothing here was seen on a wire or in the lab.

Short answer

All five builds have a sword hit test and a damage source. None of them is without combat.

- **2.04 is 2.05 rev 2.** The sword judge, the reach box, the hurt routine, the can-be-hurt test, the heart change, the hurt packet handler (ids 40 and 46), the sword-power getter and the no-kill flag test are the same code as 2.05 rev 2. Only the field offsets move (the class layout changes listed in `build-diff-204-205.md`). The 2.05 rules apply to 2.04 as they stand.
- **2.03 differs from 2.04 in three places**: the no-kill flag sits on the player, not the level; the hurt packet has no per-player field test; and neither does the props path that leads to the sword judge.
- **2.01 and 2.02 differ from 2.03 in one more place**: the sword power is stored without the offset and limit, so it is 0 to 63 and never negative.
- **2.00 is the oldest shape.** No kick test, no shared can-be-hurt test in the sword judge, no `playerhurt` event, no id 46, and a hurt packet (40) with only the hurtable, guild and range gates.

The same in every build (2.00 to 2.05 rev 2)

- **Who decides.** The victim's client judges another player's swing from that player's props, as in 2.05.
- **Swing.** The attacker's animation step is 9 to 13 (sprite S 6 to 10).
- **Reach box.** The victim must be 0 to 4 tiles ahead of the attacker along its facing (both ends count) and strictly under 2 tiles to the side, measured between the two players' X/Y props with the same (1.5, 2) centre

offset. The box test is skipped while the victim is in hurt mode (mode 3). The code is the same in all six builds.

- **Damage.** The attacker's sword power, as the victim last received it in SWORDPOWER (prop 8), in half hearts.
- **Wire form of prop 8.** A byte below 10 is the power (1 to 4 are the default swords, with their images). From 10 the power is the byte minus 30, followed by the image name.
- **Victim gates in the sword judge.** Victim AP not exactly 100; the attacker's `(tag)` not in the victim's nickname (the same bracket test as 2.05); two attacker flags clear, the same two flags 2.05 tests, which `combat.md` maps to STATUS 0x02 and 0x20 (that the 2.00 to 2.04 flags carry the same STATUS bits is inferred from their place in the test).
- **Hurt mode.** Mode 3 for 40 ticks (2 s), started for every accepted hit, with the knockback vector normalised.
- **Sword knockback direction.** Where the decompile keeps the box test's call arguments (2.00 and 2.02), the knockback handed to the hurt routine is the victim's centre minus the attacker's point, halved; the hurt routine then normalises it to length 1 (straight down when it is zero). In 2.01, 2.03, 2.04 and 2.05 rev 2 the decompile drops those arguments, so they were not compared.
- **Heart change.** A heal (negative change) needs AP at or above the same threshold expression as 2.05 (40). The paths that block it (swimming, a paused or editor state outside a sparring level) read the same.
- **Hurt packet (40).** Knockback `(byte - 0x60) / 16` tiles, power `byte - 32` half hearts, capped at one heart, and the attacker within 10 tiles (squared distance at most 100).
- **`noplayerkilling`** is in the GS1 command table of every build, and each build has a no-kill test in the sword judge (where it lives differs, below).

What changes, by build

Rule	2.00	2.01	2.02	2.03	2.04 = 2.05 rev 2
Kick test (no hit while the attacker plays `kick`)	absent	present (by animation name `kick`)	present	present	present
Victim test in the sword judge	its own: not hurt, not swimming, hearts above 0, play state exactly 0	the shared can-be-hurt test: as 2.00, plus a paused/editor test, and a non-zero play state is allowed in a sparring level	shared	shared	shared
Sword power store	the received power as is, in 6 bits (0 to 63)	as 2.00	as 2.00	power + 20, limited to -20..20	as 2.03
Negative sword power	cannot occur: it wraps in the 6-bit store (inferred from the store's mask)	as 2.00	as 2.00	heals at AP 40 or more	as 2.03
No-kill flag set by `noplayerkilling`	on the player, cleared when a level loads	on the player	on the player	on the player	on the level, with a tamper check
`playerhurt` script event on a hit	absent	present	present	present	present
Hurt packet id 46 (`PLO_HITOBJECTS`)	not handled	handled	handled	handled	handled
Hurt packet: AP 100 immune	no	yes	yes	yes	yes
Hurt packet: no-kill flag	no	yes	yes	yes	yes
Hurt packet: once per attacker per second	no	yes	yes	yes	yes

Rule	2.00	2.01	2.02	2.03	2.04 = 2.05 rev 2
Hurt packet: guild test	yes	yes	yes	yes	yes
Hurt packet: per-player field equal (meaning not traced)	no	no	no	no	yes
Props path to the sword judge: same per-player field	no	no	no	no	yes

Notes on the table:

- **2.00's victim test.** 2.00's sword judge does not call the routine the other paths use. It requires the play state to be exactly 0, so a player in a sparring level with another state is not hit by a sword, and there is no separate paused/editor test. The hurt routine it calls does have the sparring exception.
- **The power store.** The per-player value is kept in a 6-bit slot in every build. In 2.00 to 2.02 the received power goes into it unchanged, so a power below 0 or above 63 does not survive (a wire byte of 10 to 29 gives a negative power). In 2.03 and later the power is limited to -20..20 and stored plus 20. This is the store; how a real 2.00 to 2.02 attacker's negative power looks to a victim was not seen.
- **No-kill.** In 2.00 to 2.03 the command sets a flag on the local player that is cleared on level load, and the sword judge and the hurt packet test it. In 2.04 and 2.05 the flag is on the level (and is read through a tamper-checked byte). Both end when a new level loads, so they behave the same for one player in one level.
- **Once per second.** In the builds read here the per-attacker second-of-day stamp is in the hurt packet path. It was not found in the sword judge of any build (2.00 to 2.05 rev 2). `combat.md` puts it on the sword; that placement was not confirmed by this read.

Confirmed, inferred, not tested

- Confirmed from the program: everything in the table and in "The same in every build", except the two items marked inferred.
- Inferred: that the two attacker flags are STATUS 0x02 and 0x20 in 2.00 to 2.04; that a negative power wraps in 2.00 to 2.02 (from the store's bit mask).
- Not read: 2.00 revs 1 to 3, 2.02 revs 1 and 2, 1.42 and earlier.
- Not tested in the lab or on a wire.

`client-internals/damage-sources.md`

Damage from bombs, arrows, thrown objects and baddies (2.05)

How many hearts each non-sword source takes from the local player, and when it hits. Read from the 2.05 program. The sword rule is in `decomp-findings-2026-10-24-d/01-sword-damage.md`.

Each source below calls the same hurt routine with a power in hearts and a knockback direction. That routine takes the hearts off, starts hurt mode (40 ticks), and runs `playerhurt`. It does nothing while the player cannot be hurt: already in hurt mode, dead, swimming, or in the editor or paused, as in the sword notes. So overlapping pieces of one blast take hearts only once.

Powers

source	hearts taken
explosion piece of type 1 (`putbomb 1`, `putexplosion`, the blast of a fireblast or nukeshot)	0.5
explosion piece of type 2 (`putbomb 2`, the super bomb)	0.5
explosion piece of type 3 (`putbomb 3`, the jolt bomb)	1
arrow (`shootarrow`)	0.5
fireball (`shootfireball`)	1
fireblast (`shootfireblast`)	0.5, then a blast
nuke (`shootnuke`)	1, then a blast
thrown object (`waspelt`)	1, or 0.5 when the player has exactly 1 heart left
baddy touching the player	0.5, or 1 for the golden warrior (type 7)

Bombs and explosions

- `putbomb power,x,y` makes a bomb with that power. Powers 1 to 3 are the bomb, super bomb and jolt bomb of the built-in weapons.
- When it goes off:
  - Power 3 plays `thunder.wav`, and any other power plays `bomb.wav`.
  - Power 2 makes five blasts: four of reach 2 at the four diagonal corners, 2 tiles out, and one of reach 4 in the middle. All five are type 2.
  - Any other power makes one blast of reach 2, with the bomb's power as its type.
- A blast of reach `n` (at most 15) is a cross: a centre piece, then `n` pieces in each of the four directions. The last piece of each arm uses the end picture.
- `putexplosion radius,x,y` makes one blast of type 1 with the given reach.
- A piece hurts the player when its centre, offset by one tile, is less than one tile from the player's centre (x + 1.5, y + 2) on both axes. The knockback is the piece's direction.

Projectiles

- The four shoot commands make a flying object of type 0 (arrow), 1 (fireball), 2 (fireblast) or 3 (nuke), at speed 1, and play `arrow.wav`.
- Types 1 and 3 take 1 heart and the others take 0.5.
- After hitting, types 2 and 3 make a blast of reach 2 and type 1 at the hit.

Thrown objects

- A thrown object hurts the player when it lands within 2 tiles of the player's point (x + 0.5, y + 1) on both axes.
- It takes 1 heart, except when the player has exactly 1 heart left, when it takes half a heart.

Baddies

- A baddy that touches the player takes half a heart. The golden warrior takes 1.
- Touching means the player's centre is within one tile of the baddy's point on both axes.
- On a hit, the baddy may say its "when it hurts the player" verse.

Who can hurt whom

- A bomb, blast, arrow or thrown object can belong to another player. If it does, and the level has run `noplayerkilling`, it does not hurt.
- A source that belongs to no player, such as a baddy's or the level's own, always hurts.
- When a player-owned source leaves the player at 0 hearts or fewer, the client reports the owner as the killer.

Other players' bombs, blasts and shots

How the 2.05 client takes the server's copies of another player's bomb (`PLO_BOMBADD`, 11), explosion (`PLO_EXPLOSION`, 36) and shot (`PLO_ARROWADD`, 19), and when the result hurts the local player. Read from the 2.05 rev 2 program. The bytes follow the sender's player id; every value below is `byte - 32` unless it says otherwise.

Bomb (11)

- Needs at least 4 bytes: `x×2`, `y×2`, power, fuse.
- Power is `v & 7`; bit 8 marks the bomb as the sender's. The fuse is the byte itself, in ticks.
- The bomb goes through the same routine as a local `putbomb`, so water under `(x + 1, y + 1)` makes a splash and no bomb. Nothing is sent back.
- It is put in the sender's level (the local player's level when the sender is unknown).
- It burns and goes off exactly like a local bomb (60-tick fuse for a local `putbomb`, the sender's copy 5 ticks shorter), with the sender as the owner of every piece.

Explosion (36)

- Needs at least 4 bytes: reach, `x×2`, `y×2`, power.
- Power is `v & 7`, 0 read as 1 and anything above 3 read as 3; bit 8 marks the explosion as the sender's.
- It makes one cross of that reach (at most 15) and that type at `(x, y)`. The five-cross split of the super bomb belongs to a bomb's bang only, not to this packet.
- It plays one of two sounds, chosen by whether the power is 3. The two names are not visible in this routine; that they are `thunder.wav` and `bomb.wav`, as for a bomb, is an inference.

Shot (19)

- Needs at least 5 bytes: `x×2`, `y×2`, flags, sprite, type. The type byte is `byte - 33`.
- Flags: `& 3` is the direction, 4 first removes a shot this client already has, 8 marks the shot as the sender's.
- The byte before the type is the sprite id, not the type. Arrows use sprites 107 to 110 (107 + direction).
- The shot flies 1 tile per tick along its direction.
- With flag 4, the client first deletes every shot it holds that flies the opposite way with the same sprite (any arrow sprite matches any other), on the same line. That this is the copy of a shot a shield sent back is an inference.
- In a level that has run `noplayerkilling`, the receiver marks the shot as a player's shot, and such a shot never hits the local player.

When a piece hurts the local player

Each tick of its 11-tick life, a piece hurts when all of these hold:

1. the piece is in the player's level;
2. the player can be hurt (see the sword notes);
3. the centre test of "Bombs and explosions" above passes.

Then the ownership test: the piece counts as a player's when the client is online, the piece has an owner, and that owner is still in the client's player list. A player's piece in a `noplayerkilling` level does not hurt. A piece whose owner has left counts as nobody's, and it hurts.

A hit takes half a heart, or a whole heart for a piece of type 3, and knocks the player along the piece's arm. When a player's piece leaves the player at 0 hearts or fewer, the client reports the owner as the killer.

When a shot hurts the local player

Each tick the shot moves first, then:

- **Level edge.** Past -2 or at 64 and beyond in its direction, it moves to the neighbouring level of the loaded gmap block, or it is gone.
- **Hit test.** It is skipped while the shot is bouncing back, when it is a player's shot (the local player's own, or one received in a `noplayerkilling` level), or when the player cannot be hurt.
  - The shot's size comes from the sprite table entry `v` (array 24): width `((v >> 6) & 63) / 16` and height `(v & 63) / 16` tiles.
  - It hits when `x + w > px + 0.5`, `x < px + 2.5`, `y + h > py + 1` and `y < py + 3`, where `(px, py)` is the player's position.
- **Shield.** Before hurting, a shield can stop the shot. This needs all of:
  - the player faces the shot;
  - the player's shield power is above 0;
  - the shot is of type 0 or 1;
  - the player's animation is one of two names from the built-in animation list. That these are `idle` and `walk` is an inference.

  The result depends on the shield power:
  - Power 1: an arrow sprite turns back for 3 ticks with a sound. Any other sprite is gone.
  - Power 2 or more: the shot is sent back the other way, with its sprite turned, and the client sends it again as a new shot.
- **Hurt.** Otherwise the shot hurts. It uses the same ownership and `noplayerkilling` test as a piece, and the same killer report. It takes half a heart, or a whole heart for type 1 or 3. The knockback the shot passes is not visible in this decompile.
- **After a hit.** The shot is gone. A shot of type 2 or 3 leaves a blast of reach 2 and type 1, owned by the shooter, at the hit.

Open

- **Walls.** In the decompiled flow, types 0 and 1 skip the blocking test that stops types 2 and 3, so arrows and fireballs would fly through walls. The blocking test lost its arguments in the decompile, so this is not settled. Check it in the lab.
- **Baddies.** The baddy touch above was not re-read here.
- **The two shield animation names.** See "Shield" above.

Confirmed, inferred, not tested

- Confirmed from the program: the powers in the table, the bomb split, the blast shape, the projectile types, the hit boxes and the ownership rule.
- Inferred:
  - that bomb powers 1 to 3 are the bomb, super bomb and jolt bomb (from the built-in weapon scripts);
  - that the noplayerkilling check is the level flag the sword gate uses.
- Also confirmed from the program: the receive side of ids 11, 36 and 19, the owner and `noplayerkilling` test, the killer report, and the shot's hit box. The inferences there are marked in place.
- Not tested in the lab or on a wire.

`client-internals/status-bit-0x20.md`

STATUS bit 0x20: the sword is hidden (2.05)

The player STATUS property (prop 18) carries bit 0x20 while the `hidesword` timer is running. Read from the 2.05 program.

How the bit is set

- The script command `hidesword n` takes one number. It is clamped to 0..30, multiplied by 20, and stored as a tick count (the game tick is 1/20 s, so `n` is in seconds).
- Each game tick, while the count is 0 or more, it goes down by one. It reaches -1 one tick after it reaches 0.
- When STATUS is built for sending, bit 0x20 is set when the count is 0 or more.
- The count starts at -1 when the player object is made, and is set back to -1 when the player is reset.

`hideplayer n` works the same way, with the same clamp and scale, but on its own counter, which drives bit 0x02 (hidden). In the command table, `hidesword` and `hideplayer` are entries 59 and 69. Both take one number argument (`R`).

What the bit does

- Another player whose STATUS has 0x20 set (or 0x02) cannot hurt with a sword swing. The hit gate checks the attacker's STATUS (see `decomp-findings-2026-10-24-d/03-hit-gates-and-open-points.md`).
- In the local client, the counter is read only to build STATUS and to count down. Nothing else reads it, so `hidesword` does not stop the local swing itself. The swing is only refused a hit at the victim's end.

Confirmed, inferred, not tested

- Confirmed: the command, the clamp and scale, the countdown, and the STATUS bit.
- Inferred: that a client also leaves out the sword picture of a player with bit 0x20. The drawing code was not traced.
- Not tested on a wire or in the lab.

`client-internals/item-drops.md`

Item drops from bushes, pots and stones, per client version

Date: 2026-10-30

What a cut or lifted bush, a lifted pot and a stone leave, who rolls it, and what the client sends about items on the floor. Covers every build from the 4th alpha to 2.31, read in the binaries and checked in the lab with real 1.39, 2.05 rev 2 and 2.22 clients. This corrects parts of `board-respawn.md` (see the end).

Tags: **[bin]** read from the binary · **[live]** seen in the lab with the real client under Wine · **[cap]** read from a lab capture · **[src]** GServer-v2 source · **[doc]** `newfeatures*.txt` · **[inf]** inferred.

The captures, their decoded message lists and screenshots are in the project files under `graal-decompile/live-trace/item-drops/`.

Summary

Builds	Who drops	What the client sends
4th alpha to 1.37	The client rolls **[bin]**	Roll: UDP 19 to the other players only. Pickup: applied locally, UDP 20 and TCP `PLI_ITEMDEL` (13). Timeout: nothing **[bin]**
1.38 to 1.41	The client rolls **[bin]**	Roll: UDP 19 and TCP `PLI_ITEMADD` (12). Pickup online: `PLI_ITEMTAKE` (32), and the server gives the item with props. Timeout online: `PLI_ITEMDEL` (13) **[bin][cap]** for 1.39; 1.41 read for the roll only
1.42 (NewMain) to 2.31	The server (`PLO_ITEMADD`, 22). Offline nothing drops **[bin][doc]**	Pickup: `PLI_ITEMTAKE` (32). Timeout: `PLI_ITEMDEL` (13) **[bin][cap]**

- **The change is at 1.42, not 2.00.** NewMain 1.42's `newfeatures142.txt` says "bush rupees and pot hearts are server-side", and the roll is gone from NewMain 24 and 26 **[doc][bin]**. 1.41 (graal_gr1411) still has it **[bin]**.
- **No client ever rolls arrows**, and no client drops anything for a stone **[bin]**. Arrows from a bush come only from a server that has them in its table (GServer-v2: `rand() % 6` includes darts **[src]**).
- **Stones.** The only client path that touches a stone is the bomb blast, which destroys a sign, pot or stone and gives a heart only for the pot **[bin]**. "Something with rocks" is most likely bombs breaking them.

The 1.x roll [bin]

One `Random(100)` per drop, with Delphi's `Random` (`seed = seed * 0x08088405 + 1`, result `(seed * range) >> 32`):

Roll	Item
0 to 24	green rupee (25 %)
25, 26	blue rupee (2 %)
27 to 31	heart (5 %)
32 to 36	bombs (5 %)
37 to 99	nothing (63 %)

- The same table is in every build from the 4th alpha to 1.41. From 1.38 the code picks the item by its number in the item table (0, 1, 5, 3) instead of by name.
- **Sword cut:** every entry of the cut table rolls, tall grass too (1.3 and 1.41 read). `board-respawn.md` said bush only.
- **Lift:** lift entry 0 (bush) rolls; entry 2 (pot) always leaves a heart; sign and stones leave nothing.
- **Bomb blast** (`TAdventureExplosion`, four test points): cut entry 0 rolls; lift entries 1 to 3 (sign, pot, stone) are destroyed, and the pot leaves a heart.
- The item lies at the object's top-left tile (1.39 capture: the bush at 31, 34 sent `^d`).

- A new item gets life 164 and the "announce" flag.
- `bushitems` in GServer-v2 beta4's `serveroptions.txt` says the same thing from the server side: "If making a 1.41 server, set bushitems, vasesdrop, and baddyitems to false as the 1.41 client generates items."

Items on the floor (2.05 rev 2) [bin]

The item object (`TAdventureExtra`) and the level's item tick:

- **Life.** Every 50 ms tick the life goes down by 1, from 164. Below 0 the item goes; online the client sends `PLI_ITEMDEL`. Lab: 8.20 s from the drop to the 13 **[cap]**.
- **Pickup** when all hold: the item is not permanent (life below 100 at creation, such as a chest's display item with life 24), the life is below 160 (so not in the first four ticks), and `|px + 1.5 - (ix + 1)| < 2` and `|py + 2 - (iy + 1)| < 2`.
- At life 140 the `playerlaysitem` event fires.
- **Blink:** not drawn while the life is below 37 and odd.
- **Bounce:** raised by `|sin(life / 3)| · (life / 164)^16 · 3` tiles while the life is at least `164 - 7π`.
- **Draw:** the shadow, sprite 42, at `(x + 0.5, y + 1 + h / 2 - 0.5)` for an item `h` tiles high; the item centred on `(x + 1, y + 1)`.
- **Sprites by item number:** 136, 137, 138, 67, 143, 139, 145, 351, 65, 350, 314, 315, 321, 323, 322, 324, 325, 328, 329, 330, 331, 332, 333, 334, 349.
- **Floor items** (the announce flag; `PLO_ITEMADD` makes an item only for these): 0 to 5, 7, 8, 19 and 24 (rupees, bombs, darts, heart, bow, bomb, gold rupee, spin attack).
- **What a pickup gives offline:** green 1, blue 5, red 30, gold 100 rupees (up to 9,999,999); bombs and darts 5 each (up to 99); a heart one whole heart. Sound `extra.wav` (1.3 and 1.39: `sounds\extra.wav`).

1.3 differs **[bin]**: the pickup test is an overlap of pixel boxes, and a timeout is silent. Whether 1.3's life count stops behind a flag (offset 0x74) was not settled.

Lab runs [live][cap]

Client and server	What happened
2.05 rev 2, old lab pack (2.05 in the 2.1 slot, `tiledroprate` 100)	Cut: C>S 1, then S>C 7 and S>C 22 `^d"` (red rupee at 31, 34). Left alone: C>S 13 `^d` 8.20 s later. Picked up: C>S 32 `^d!`, then S>C 9 RUPEESCOUNT 5, and the HUD showed 5. A lifted pot: S>C 22 heart
2.22, beta4	The same: 22, then 13 after about 8.7 s; pickup 32, then props 9
2.05 rev 2, beta4	No drops: beta4 counts 2.05 as `CLVER_2_052`, below 2.1, and GServer-v2 gives no drops below 2.1 **[src]**
1.39, beta4 (`generation = classic`)	The client rolled: C>S 1, then C>S 12 `^d` in the same frame. Picked up: C>S 32 `^d`. The server did not echo 22 to the sender. Four of the next five cuts dropped nothing

Seihai against the same servers (2026-10-30) [cap]

- **2.05:** cut, S>C 22 bombs, drawn with its shadow and bounce; C>S 13 8.39 s later. A heart: picked up after 1.09 s with C>S 32, S>C 9 CURPOWER.
- **1.39:** lift, C>S 1 and C>S 12 `^\x82` (green rupee) in the same frame; C>S 32 on pickup; S>C 9 RUPEESCOUNT, and the HUD count went up.

Corrections to `board-respawn.md`

- Drops became server-side at **1.42**, not 2.00.

- The 1.x cut roll is for every cut entry, not the bush alone.
- From 1.38 the roll also goes to the server (`PLI_ITEMADD`), not only to peers.
- The drop tables of 1.39 to 1.41 were read: the same as 1.3.

Open

- 1.41's pickup and timeout path is assumed to be 1.39's.
- 1.x pickup box in tiles (only 1.3's pixel overlap was read).
- Whether items are drawn under or over players (Seihai draws them under, inferred).

`client-internals/board-respawn.md`

Bush, pot and stone respawn: who restores the board, when, and who drops items

Date: 2026-10-30

How cut bushes and grass, lifted bushes, signs, pots (vases) and stones, and other board changes come back, and where the rupees and hearts they drop come from. Covers the 1999 GServer13 (shipped with 1.3 test), the 1.3 test client, the 2.05 rev 2 client, and the GServer-v2 lab server.

Tags: **[bin]** read from the binary · **[live]** measured with the real program under Wine (`tools/respawn-lab/`) · **[cap]** earlier lab capture (`protocol/packet-layouts.md`) · **[src]** GServer-v2 source · **[doc]** the official `newfeatures*.txt` / `readme.txt` · **[inf]** inferred.

Summary

	1.3 era (GServer13 + 1.3 client)	**2.00 to 2.05 (client)**	**GServer-v2 lab server**
Who restores a cut or lifted object	Server only	Server only	Server only
Clients regrow anything on their own	No	No, offline nothing ever comes back **[live]**	n/a
Which changes respawn	Any change that is exactly **2 × 2** **[bin][live]**	n/a	A change whose top-left tile was one of 11 listed ids **[src]**
Timer	`trunc(R·rand·0.5 + 0.75·R)` one-second ticks, R = `respawntime`, default **10**, so 6 to 12 s **[bin][live]**	n/a	Exactly `respawntime`, default **15** **[src][cap]**
Restore packet	`PLO_BOARDMODIFY` (7) with the old tiles, to everyone in the level	handled as any board modify	same
Sender gets its own change echoed	No **[bin][live]**	n/a	Yes, to the whole level **[cap]**
Item drops from bushes and pots	**Client**, local random roll, sent peer to peer, and from 1.38 also to the server (`PLI_ITEMADD`) **[bin] [cap]**	**Server** ("bush rupees and pot hearts are server-side", NewMain 1.42 **[doc]**). The 2.05 cut and lift code spawns no item **[bin]**	Server, 50 % of 6 items for bushes, a heart for a vase **[src]**

The objects

The tables are `arrays.dat` lists. They are the same in 1.21, 1.3 test and the 2.22 install (2.05 runs from that folder) **[bin]** (`tools/respawn-lab/arrays_dump.py`). In 1.21 they sit one index lower because 1.21 has 24 arrays, not 25.

Cut with the sword:

#	Object	Tiles (2 × 2, top row then bottom row)	Leaves
0	bush	2 3 / 18 19	677 678 / 693 694
1	tall grass	420 421 / 436 437	679 680 / 695 696

Lift with A, in glove order (name table in the 2.05 exe: `bush sign vase stone blackstone`):

#	Object	Tiles	Leaves
0	bush	2 3 / 18 19	677 678 / 693 694
1	sign	512 513 / 528 529	1802 1803 / 1818 1819
2	pot (`vase`)	684 685 / 700 701	1770 1771 / 1786 1787
3	stone	34 35 / 50 51	1834 1835 / 1850 1851
4	black stone	990 991 / 1006 1007	1834 1835 / 1850 1851

Glove power g allows the first g + 2 lift entries (`decomp-findings-2026-10-25-l/03-cut-lift-chest.md`). Players start at glove 1, so the pot lifts with no glove item.

A thrown object does not touch the board when it lands. It breaks into leaves (`TAdventureLeap`, `crush.wav`, or `water.wav` for entry 5). The board change happens once, at the lift. Respawn therefore puts the object back where it was lifted, not where it landed.

"Swamp" is not a destructible object. It is tile type 6 (and 7, lava swamp) in the tile-type list (`newfeatures2001.txt`, `setshape2`). GServer-v2's comment "Bushes, grass, swamp" refers to its own tile list, not to a third cut type.

Push and pull blocks are different again. The 2.05 client sends the 4 × 4 area around the old spot (`ledges-push-pull.md`). GServer13 never respawns that (see below).

Clients

No client-side respawn

2.05 rev 2, offline, live. Level `resp1.nw` (made with `tools/respawn-lab/mklevel.py`) was played with the real client under Wine, and both 64 × 64 boards in its memory were polled every 10 ms for 10.5 minutes (`boardwatch.py`).

- Sword on a bush: tiles 2 3 18 19 became 677 678 693 694 at once.
- A on a pot: 684 685 700 701 became 1770 1771 1786 1787.
- A second bush was cut later.
- No tile changed back in the whole run, 633 s after the pot was lifted.

The client has a `TBoardModify` class, but it is only a record built to send one change (`x, y, w, h`, tile array). It has no timer **[bin]**. The 1.3 client is the same: its `TBoardModify` users are the cut and lift sender, the push and pull code, and one other board-write path, and none of them keeps a timer **[bin]**.

So a cut bush stays cut until the level is loaded again. Whether 2.05 offline reloads the level from disk on re-entry was not tested here.

Sending the change

- **2.05:** cut and lift both call one routine. It writes the leftover tiles to the board, then, only when online, sends `PLI_BOARDMODIFY` (1) with `c x, c y, c w, c h` + w × h GSHORT tiles. It sends nothing if the tiles were already equal. Cut then adds the leaves effect **[bin]**.

- **2.04+ (2.05 checked):** the whole 2 × 2 has to match a table entry before anything happens. 1.3 matched the single tile under the sword tip and wrote the leftover block at the derived corner **[bin]**. This is the 2.04 change "check for correct tiles when lifting/eating/slaying bushes" **[doc]**.
- **2.05:** "bombs don't destroy half bushes" **[doc]**.

Item drops in 1.3 (client-side)

Drops are rolled by the client that cut or lifted the object **[bin]**:

When	Drop
Any cut entry (bush and tall grass), a bush lifted, or a bush destroyed by the bomb blast (cut entry 0) (corrected in `item-drops.md`)	`Random(100)`: 0–24 green rupee (25 %), 25–26 blue rupee (2 %), 27–31 heart (5 %), 32–36 bombs (5 %), 37–99 nothing (63 %)
Pot lifted, or destroyed by that blast routine (lift entry 2)	A heart, every time

- **Timing:** the drop happens at the lift, not when the thrown pot breaks.
- **Item flags:** items are created with flag value 164 and the "announce" flag set.
- **When online:** the client sends the new item straight to the other players over UDP (`sendto` each peer), as id 19, with `chr(x·2+32) chr(y·2+32) chr(item+32)`. Only item kinds marked in a per-item table are sent. The GServer does not take part. The board change itself still goes to the server over TCP (id 1).

Item drops from 1.42 on (server-side)

- **1.42:** "bush rupees and pot hearts are server-side" (`newfeatures142.txt`; the roll is gone from NewMain 24 and 26, and 1.41 still has it) **[doc][bin]**. This note said 2.00 before; see `item-drops.md`.
- **2.05 binary:**
  - The cut routine creates only `TAdventureLeap`.
  - The lift routine creates only the carried object.
  - The only `TAdventureExtra` constructor is reached from the chest code, a weapon-drop path, one network handler (most likely `PLO_ITEMADD` **[inf]**) and the GS1 executor. None of these is the cut or lift code.
  - So **an offline 2.05 bush or pot gives nothing**, and online every drop is a `PLO_ITEMADD` (22) from the server **[bin]**. The capture in `packet-layouts.md` shows the same thing: no client sent `PLI_ITEMADD` and the server sent the drops **[cap]**.
- **Not covered:** a 2.x-era official GServer binary is not in the archive. The real 2.x drop rates are unknown, and only GServer-v2's rates are written down.

GServer13 (November 1999, GSERV014)

Options

`serverconfig.txt` keys and the start dialog (screenshot-checked, defaults filled in):

Key	Dialog label	Default
`respawntime`	Respawn time (bushes etc.) in sec	10
`horselifetime`	Horse lifetime in sec	10
`apincrementtime`	Alignment increment time in sec	1200
`baddyrespawntime`	Baddies respawn time in sec	60

GServer 1.28 beta 2 (`GServerv.exe`) has the same four keys.

`PLI_BOARDMODIFY` (1) handling [bin]

1. Needs at least 4 bytes and a current level, and `len ≥ 4 + 2·w·h`.
2. Writes the new tiles into the server's 64 × 64 board and builds the old-tile string from what was there.
3. If any tile changed:
    - Relays `PLO_BOARDMODIFY` (7) with the client's bytes to every other player in the level. **The sender is skipped.**
    - Makes a change record: new tiles, old tiles, a countdown and a reference count.
    - **If w = 2 and h = 2:** countdown = `Trunc(Random · R/2 + 3R/4)` with R = `respawntime`, and the record joins the level's respawn list.
    - **Any other size:** countdown −1, so it never respawns. This covers 1 × 1, 2 × 1, 3 × 3 and the 4 × 4 push/pull block.
    - The record is also added to the "changed since you left" list of every player who has left this level since loading it. Any earlier record with the same `x, y, w, h` header is replaced.
4. **No tile check.** Any 2 × 2 change respawns, whatever the tiles. A client can blank a 2 × 2 of floor and it comes back.

Respawn tick [bin]

- `Timer1` (TTimer default interval, 1000 ms) walks every loaded level, empty or not, and decrements each countdown.
- At < 1, the server:
    - removes the record;
    - sends `PLO_BOARDMODIFY` (7) with the **old** tiles to everyone in the level, the original sender included;
    - writes the old tiles back into its board;
    - updates every "changed since you left" list. If the list still holds the original change, it is dropped, because that player never saw it. If not, the restore is queued instead.
- **Timing:** the countdown starts at an integer n in 7..12 for R = 10, and the first tick comes 0 to 1 s after the change. The restore therefore lands n − 1 to n seconds later.
- **Weights:** n = 7 and n = 12 are half as likely as 8 to 11, because of the truncation of [7.5, 12.5).

Measured [live]

GServer13.exe ran under Wine in a loopback-only namespace with one load-only account. Two scripted clients (`g2cli.py`) were used: A sends changes and B watches.

Test	Result
2 × 2, 60 positions, one every 0.37 s	All 60 respawned. Delays 6.21 to 11.98 s, mean 9.21 s. ceil(delay) 7/8/9/10/11/12: 4, 9, 16, 11, 12, 8 (expected 6, 12, 12, 12, 12, 6).
1 × 1, 2 × 1, 1 × 2, 3 × 3, 4 × 4	Relayed to B at once, never respawned (watched 20 s past the 2 × 2 respawns)
Echo	A never received its own change. A and B both received every respawn.
Restored tiles	The level's original tiles (632 for `house1.graal` floor), not a fixed "bush" id

Raw rows: `version-profiles/lab-gs13-respawn.tsv`.

Level re-entry [bin]

- **On leave:** the server starts a per-player change list for that level.

- **On re-entry:**
  - If there is no list, the client gets the level as on a first visit.
  - Otherwise only the queued changes go out (as id 0), and the list is dropped. **[bin]**, the branch is hard to read, so treat the exact packet as **[inf]**.
- **Effect:** a player who leaves while bushes are cut and comes back after they regrew is sent nothing for them.
- **Disconnect:** the player's lists are freed.

GServer-v2 (lab server) [src]

This is the open reimplementation used for the 2.05 and 2.22 captures. It is not an original server.

- **Respawn condition:**
  - The top-left tile *before* the change must be one of `0x1ff 0x3ff 0x2ac 0x002 0x200 0x022 0x3de 0x1a4 0x14a 0x674 0x72a`. Those are bush (2), tall grass (420), pot (684), sign (512), stone (34), black stone (990), and five others.
  - Then the change respawns after exactly `respawntime` (default 15).
  - Otherwise it is kept with no timeout.
- **Size:** any size can respawn.
- **Overlap:** a new change deletes stored changes it fully covers.
- **Echo:** relays `PLO_BOARDMODIFY` to the whole level, sender included.
- **Restore:** swaps old and new tiles. It keeps the record, so late joiners get the restored state.
- **Drops:** only for clients at the 2.1 version slot or above.
  - **Bush, grass or swamp tiles** (2, 0x1a4, 0x1ff, 0x3ff) with `bushitems`: `tiledroprate` % (default 50) of the time, item `rand() % 6`.
  - **Pot** (0x2ac) with `vasesdrop`: a heart.
  - **Why 2.05 got drops in the lab:** the lab maps `GNW03081` (2.05) into the 2.1 slot (`tools/lab-setup.md`), which is why the earlier capture shows server drops for 2.05.
- **Push-pull blocks:** a 4 × 4 change made of a red or blue push-pull stone is refused and the board is sent back when `clientsidepushpull` is on.

What a Seihai version profile should do

Profile	Respawn	Drops
1.x online (GServer13-style)	Server, any 2 × 2, `trunc(rand·R/2 + 3R/4)` s on a 1 s tick, R = 10	Client rolls the 1.3 table (above) on any cut and on a lifted or blasted bush, a heart for every pot. Announce to peers; from 1.38 also `PLI_ITEMADD` to the server (`item-drops.md`).
1.x offline	None	Same client table
2.00–2.05 online	Server pushes `PLO_BOARDMODIFY`. Client never regrows.	Only `PLO_ITEMADD` from the server
2.00–2.05 offline	None	None
Seihai's own server	Pick per profile: GServer13 (random 0.75–1.25 × R, 2 × 2 only, no echo) or GServer-v2 (fixed R, tile whitelist, echo)	

Open

- The real 2.x GServer's timer, tile whitelist and drop rates. No binary of a 2000–2002 official server has been found.
- Whether 2.05 offline reloads a level from disk on re-entry, which would make a cut bush come back.

- Drop tables in 1.21: same routine shape expected, not read. 1.33 to 1.41 were read in `item-drops.md`: the same table.
- Later clients (2.2+ `updateboard2` / `PLO_BOARDMODIFY2` on gmaps, v4+). Not covered here.

Lab notes

- **2.05 offline:** `tools/respawn-lab/mklevel.py` writes a level with every object in a row. `boardscan.py` finds the board copies in client memory. `boardwatch.py` logs every tile change with a timestamp. Keys: `xdotool keydown Up`, `s` for sword, `a` for lift. The Wine screen often stops repainting tiles after a few actions, so trust the memory log.
- **GServer13:**
  1. Extract `graal13b.7z` and start `GServer13.exe` under Wine inside the namespace.
  2. Press OK on the options dialog.
  3. Edit accounts, then Guest Accounts, then Add: name `guest`, password `guest`, load only, level `house1.graal`.
  4. Run `gs13_respawn.py out.jsonl 60` inside the namespace.

`client-internals/feature-specs/04-death-respawn.md`

04 Death, respawn and health regeneration

Snapshots read: Graal_analysis `8da6800`, Seihai `06aacfd`, GServer-v2 `TPlayerProps.cpp`, `TPlayer.cpp`, `TAccount.cpp`.

What it is

Death is **client-driven**. When hearts reach zero the client sets the DEAD bit in its STATUS prop (bit `0x08` of prop 18), sends the prop, and (if a killer is known) sends `PLI_CLAIMPKER` (14) `s killer id`. The server reacts to the bit transitions; it does not move the player or heal gradually.

Exact server behaviour (`TPlayerProps.cpp`, STATUS case) [srv]

- **Dies** (bit goes 0 to 1): unless the level is a sparring zone, `deaths` increases and `dropItemsOnDeath` runs (random gralats between `mindeathgralats` and `maxdeathgralats` as a floor item, plus items if `dropitemsdead`). If the dead player was level leader and others are present, leadership passes to the next player and that player gets `PLO_ISLEADER` (10).
- **Revives** (bit goes 1 to 0): the server sets hearts by AP: AP under 20 gives 3 hearts, under 40 gives 5, otherwise the full MAXPOWER; at least 0.5. It pushes `PLPROP_CURPOWER` to the player and the level, and re-sends `ISLEADER` if the player is leader. **There is no warp**: where the player respawns is a client decision.
- AP (the "alignment" prop 32): with `apsystem` on, `TPlayer::doTimedEvents` counts `apCounter` down once a second (not while paused or in a sparring zone). At zero AP goes up by 1 (max 100), the server pushes `PLPROP_ALIGNMENT` to the player and the level, and reloads the counter from `aptime0..4` (30/90/300/600/1200 s in the lab, by AP band 0-19, 20-39, 40-59, 60-79, 80+). Entering a sparring zone at 100 AP drops it to 99.
- `PLI_CLAIMPKER` (`msgPLI_CLAIMPKER`): ignored for self-kills or an unknown killer. In a sparring zone a Glicko-style rating update runs for both players (skipped if both share an IP) and `lastspartime` is set. Elsewhere the killer gets +1 kills (unless `dontchangekills`), and if `apsystem` is on and the victim has

20+ AP and the killer has any AP, the killer loses AP by a formula based on both AP bands and the killer's counter is reloaded.
- Heart regeneration is not periodic on the server. Hearts come from heart items (03), `healswords`, the revive rule above, or script/NPC props. A client-sent CURPOWER above the server value is refused while AP is under 40.
- Login saves/loads HP (`HP`, `MAXHP`, `AP`, `APCOUNTER`, `KILLS`, `DEATHS`) in the account file (05).
- Idle: `disconnectifnotmoved` + `maxnomovement` (1200 s in the lab) kicks idle players with a disconnect message (see 13).
- Baddy death timers are in 02 (2 s to dead, `baddyrespawntime` 60 s).

What 2.05 does on the client side [2.05][dec]

From the 2.05 rev2 decompile (2026-10-24). Tick = 50 ms. Behaviour only; tables are in `../decomp-findings-2026-10-24/` (`01-death-respawn.md`, `constants-205.tsv`, `life-states.tsv`, `player-modes.tsv`).

Trigger. Any change that leaves hearts at 0 or below starts the death sequence once. A per-frame safety net also forces the dead life-state whenever hearts are 0 or below.

Sequence on the dying client 1. A death timer starts at 0 and counts up each tick; it **stops at 30 (1.5 s)**. 2. Life-state becomes dead, mode returns to 0 (normal) and the player faces down. 3. The local `dead` gani plays and the script event `playerdies` fires. 4. **Controls are ignored until the timer passes 27 (1.35 s)**, so input works from timer 28. 5. The STATUS prop (18) gets the dead bit (0x08) **only when life-state is dead and the timer is at least 28** (about 1.4 s after death). Other players therefore see the death about 1.4 s late. 6. No dead sprite goes on the wire: prop 17 shows idle plus direction (see 11). The dead state is carried by the status bit only.

Respawn - After the timer passes 27, any movement or action key respawns the player. **There is no respawn packet and no server permission step** on this path. After respawn the client clears the status bit 0x08 when life-state returns to playing, and the server's revive rule (above) sets hearts and pushes CURPOWER. - Hearts the client sets, by AP: below 20 gives 3; 20 to 39 gives min(5, max hearts); 40 or more gives max hearts. (Same bands as the server's revive rule.) - **Destination** = the last saved level-entry point (level, x, y, direction). It is updated on every level entry (unless the old and new levels share the same "name group", see open items) and at login. If nothing was saved and the client is online: x 30.5, y 30.0, facing down, in the current level. Offline: the default start level `tutorial_b-4.graal`. - When the online flag is set the client **re-sends the current level at the current x/y**, so the server sees an ordinary level warp (`PLI_LEVELWARP` route), not a "died" message.

CLAIMPKER. The *victim's* client sends `PLI_CLAIMPKER` (14) with the attacker's player id when a sword hit, an explosion hit or a UDP hurt leaves hearts at 0 or below. The server's reaction is in `TPlayerProps.cpp`/`TPlayer.cpp` (above); it is not in the client binary.

Damage rules that feed this - Healing only happens at AP 40 or above. - No damage in swim mode (10) or in the editor/pause state unless forced. Hurt state: mode 3 for 40 ticks, `hurt` gani, knockback, script event `playerhurt`. A player already in mode 3, mode 10 or dead cannot be hurt. - Hearts, AP and max-power sit in an attribute array the client stores obfuscated twice and cross-checks; a mismatch raises "memory problems error 1/2". Editing hearts from outside the client is detected by the client itself. A Seihai that holds hearts in plain fields has no such check (fine, the server stays authoritative).

Open - The exact "same name group" rule for *not* updating the saved entry point was only partly traced. Until it is known, update the saved point on every level entry and treat a warp inside one map or group as a Seihai choice. - The server-side reaction to CLAIMPKER is described above from GServer-v2, not from the client. - The `0x20` STATUS bit is the `hidesword` timer (`../status-bit-0x20.md`); a few mode values are unlabelled (see 11). -

~~1.39 death handling~~ — filled in `../../version-profiles/matrix/rules-139.md` (same 30-tick timer and STATUS bit rule as 2.05; hearts to max, no AP bands). 1st Graal still open.

Seihai today

- Reads `player_health` for the HUD. Does not send STATUS, does not track the dead bit, does not send CLAIMPKER, and treats `ISLEADER` (10) as unhandled.
- Offline: the web-Seihai design (`formats/seihai-systems.md`) has its own death panel and revive (85% of rupees kept, spawn at level start). Seihai's own invention, fine for offline.

Steps for Claude

1. Track CURPOWER from the server. Add `dead: bool` and a death tick counter (50 ms ticks, capped at 30). When hearts become 0 or below after a hit (02), start the timer, play `dead`, and send CLAIMPKER (14) when the hit has an attacker id. Set the DEAD bit (0x08) in STATUS and send it only once the timer reaches 28 (2.05 behaviour).
2. While dead: show the `dead` gani frame (or a placeholder), block movement and weapons until the timer passes 27. Send prop 17 as idle plus direction.
3. Respawn on any movement or action key once the timer has passed 27 (the 2.05 rule, no timer-only respawn): clear the bit, send STATUS, accept the server's CURPOWER push (hearts by AP, as in the table above). Respawn location: the last saved level-entry point (level, x, y, direction), updated on every level entry; if none, x 30.5, y 30.0 facing down in the current level. Re-send the current level at that x/y as an ordinary level warp. No respawn packet exists. Keep the saved point configurable only for the open "same name group" case.
4. Show gralat loss from the floor item the server drops (ITEMADD from 03).
5. Handle `PLO_ISLEADER` (10) by storing `is_leader` (needed for 02 baddies).
6. Show an HP regen hint only if the server pushes CURPOWER; do not simulate regen.

How to test

- Lab: Seihai sends the dead bit (STATUS prop with bit 0x08) and then clears it. Check the lab account file: `DEATHS` increments, and the server pushes CURPOWER = full hearts (the default account has AP 50, so the rule gives MAXPOWER, 3 hearts). Set AP under 40 in a copy of the account to see the 5-heart and 3-heart bands.
- Second client observes `PLO_OTHERPLPROPS` with the STATUS dead bit.

NEEDS

Filled by the 2026-10-24 2.05 decompile: timings, input lock, status-bit delay, hearts by AP, destination rule, and that no respawn packet exists. Server side was already known.

Still open: the "same name group" rule for the saved respawn point, a few unlabelled mode values, and the 1st Graal death handler (1.39 is in `../../version-profiles/matrix/rules-139.md`). A lab capture of a real 2.05 client dying twice would confirm the sequence on the wire (status bit timing, the warp frame after respawn).

`client-internals/lab-captures-2026-10-25/01-damage-chest-respawn-weapons.md`

Lab captures 2026-10-25: chests, damage, CURPOWER, respawn, default weapon ids (2.22, derived facts only)

Experiments: A chests, B damage powers, C CURPOWER and packet 40, D end-of-respawn save, E default weapon ids. Verdicts are marked settled or OPEN in each section. Method: see the end of this file.

Part 0: facts decoded from earlier-attempt leftovers (labm/res/*.out, real 2.22 client c1 as victim; server->client packets injected through proxy2.py; c2 = a second real 2.22 client, other player id 3)

Harness caveat: positions are read from the client's own X/Y props; hearts are also read from screenshots.

C. CURPOWER semantics (settled for 2.22)

- CURPOWER (prop 0x22) is in HALF hearts; MAXPOWER (prop 0x21) is in hearts. Injecting MAX=10, CUR=28 then taking a half-heart hit made the client send CUR=19: the client clamps CUR to MAX*2 (20) before subtracting. MAX=3, CUR=10 then a half-heart hit sent CUR=5 (clamp to 6, minus 1); the screen showed 2.5 hearts.
- The client does not send anything when the server sets CUR/MAX (no echo).
- After a hit the client sends ONE props message: CURPOWER (new value) + GANI `hurt` + one more prop (0x31). Our own CURPOWER write on injected CUR=0: client sends `dead` GANI and prop 0x32, no CURPOWER.
- After the death key (arrow key) the client sends CURPOWER = hearts by AP bracket (MAX=10): AP 10 and 19 -> 6 (3 hearts); AP 20 and 39 -> 10 (5 hearts); AP 40, 60, 100 -> 20 (10 hearts = full). (lab result file)

B. Packet 40 (hurt player), real client as victim (settled for the cases below)

- Payload as accepted: attacker id (2 bytes), dx byte, dy byte, power byte. Hit applied only when the attacker id is a player the client knows (ids 99, 0, 2 here: ignored, no damage, no knockback).
- Power byte (byte-0x20) is in HALF hearts: 0 -> knockback only, no hearts; 1 -> half heart (CUR 6->5); 2, 3, 4, 15, 31 -> one heart (CUR 6->4). So the cap is 2 half hearts, matching the decompile ("capped at one heart").
- Knockback: direction from (dx-0x60, dy-0x60); always 5 tiles (50 ms steps); diagonal (0x70,0x70) gives (3.5, 3.5) so the vector is normalised. 0x61/0x68 gave the same as 0x70 (sign/direction only), 0x5f/0x50 went the other way.
- A trailing 4-byte field (sent as 20 20 20 20) made the hit do NOTHING (no damage, no knockback); 2 trailing bytes were still accepted, 3 were not. So the NPC-id field (when present as the 4th extra byte) is looked up: an unknown NPC id 0 drops the hit. (Open: what a real NPC id does.) The earlier distance test (exp40d) used the 4-byte form, so its "no hit at distance^2 81..132" is NOT evidence for a distance gate; re-test is in Part 1.
- Server-sent bomb (PLO 11) and arrow (PLO 19) injected near the victim: half-heart hit (CUR 6->5) with a 5-tile knockback directly away from the bomb/arrow start. Damage showed 4 (a full heart) in a few runs (bomb field 0x23, 0x2b; arrow last byte 0x24) without a clear rule: OPEN.
- Real arrow shot by c2 (last byte 0x21 on the wire) never hurt c1 in realarrow2, and an injected arrow with last byte 0x20-0x22 did nothing while 0x23-0x2c hurt: OPEN what that byte means.
- (superseded by the Part 2 bomb notes below) Bomb hit zone from the first sweep was misread.

D. End-of-respawn save (open, partial)

- After death the client sends: CURPOWER 0 + GANI dead; arrow key -> a warp request (client msg id 30) carrying (x,y,level) = (30.0, 30.5, lv_a.nw) = the level's login/entry point, not the death spot (26.0 / 23.5). Second death after the server's warp reply (with the reply delayed 8 s or not delayed): request again (30.0, 30.5). So after the reply arrives the saved point is the entry point either way.
- NOT tested: dying again inside the delay window (before the reply), which is the only case where "save at end of respawn" would show the death spot.

Part 1 (fresh runs, 2026-10-25 ~22:50-23:10 JST): A. Chests (real 2.22 client, labm lab, PLO_LEVELCHEST (id 4) injected)

- Full (closed) form `00 x y item sign` (each +32) is drawn as a closed chest, short form `01 x y` as an OPEN chest (dark inside). Each is a 32x32 px block = 2x2 tiles at x,y (top-left), drawn over the grass board; no board tile needed. Settled for 2.22.
- A closed chest is solid like a wall: walking into it from any side stops the player when the 12 body probes touch its 2x2 box (matches #61). Pushing against it plays `push.gani`.
- OPENING: only when the player faces UP (the chest is above, the player below it) and presses the action key (A). Facing down, left or right with the player stopped against the chest and A pressed: NOTHING is sent (3 sides tested, each with a chest adjacent). Settled for 2.22; 2.05 not checked.
- On opening (facing up) the client at once: asks for `chest.wav` (WANTFILE), sends PLI_OPENCHEST (20) with x,y = the chest's top-left tile (whole tiles, +32), and draws the chest open locally (no server reply needed). About 1.2 s later it sends packet 13 with x*2*,y2 = (chest.x, chest.y-1), i.e. one tile above the chest. It does NOT send a rupee/props update; the rupee counter stayed 0 (the lab server did not know the injected chest, so the reward cannot be judged here). Open: what the server sends back for a known chest.
- Compare Seihai (documentation/chest-drawing.md, #65): drawing and short/full forms agree. Difference: Seihai opens a chest from any facing (offline: the point one tile ahead of the feet); the real 2.22 client only from below facing up.

B/C addendum (fresh runs 23:10-23:30 JST): packet 40 distance gate

- Clean packet (attacker id 3, dx dy power, no trailing bytes), victim c1 on the same row, attacker c2 fixed at x=33.0: hit (CUR 6->4, knockback) at squared distance 64, 81, 90.25, 100.0; NO hit (hearts unchanged, no knockback) at 110.25 and 121.0. So the gate is distance^2 <= 100 (10 tiles), settled for 2.22. The earlier "no hit at 81" results used the NPC form and are explained by the NPC rule below.
- NPC field: the trailing field is 3 bytes (GINT3 npc id). With 3 or 4 trailing bytes (`20 20 20` = id 0, `20 20 21` = id 1, `2a 2a 2a`) the real client refuses the hit completely (no damage, no knockback); with 2 trailing bytes it still hits. This is exactly Seihai's `parse_hurtplayer` (npc only when >= 6 bytes follow the id) plus `Reject::NpcSource`.
- Zero knockback vector (dx=dy=0x60): knocked 5 tiles EAST in every run. Matches the decompile ("a zero vector is replaced by one tile east") and #55.
- Everything else in packet 40 (power cap 2 half hearts, power 0 = knockback only, unknown attacker refused, 5-tile normalised knockback) already matches Seihai #55: no Seihai change needed for packet 40.

D. End-of-respawn save (settled for 2.22, same-level respawn)

Run respawn5.py: die at x=26 (hearts 1), hold the server's reply for 9 s, press an arrow key (warp request (30.0, 30.5, lv_a.nw) at +3.1 s), then INSIDE the 9 s window reset hearts, die again at x=28.5 and press a key: the second warp request is again (30.0, 30.5, lv_a.nw), not the death spot (28.5 / 26). After the held replies arrive the

player is at (30.0, 30.5). So the respawn routine's extra save does not make the next respawn point the death spot, with or without the delayed reply. Seihai's current behaviour (respawn at the level's entry point; no end-of-respawn save) agrees. (Seihai documentation said "needs capture"; update the doc.)

E. Default weapon ids -> images (real 2.22; only what the captures show)

- Login with the server's PLO_DEFAULTWEAPON (43) frames filtered to id 7 alone: the client asks `wbowi1.png` (then `wbomb1.png`). Login with ids filtered to none, or to 8 alone: `wbomb1.png` is asked and `wbowi1.png` is NOT. So id 7 -> `wbowi1.png` is settled; `wbomb1.png` is asked regardless of id 8 (it comes with the client's own bomb item), so id 8 -> `wbomb1.png` is NOT shown by the capture.
- Ids sent after login (one per second, ids 0..39): the client asked for a file only for: 12 -> `wbomb2.png`, 20 -> `wbowi2.png`, 21 -> `wbowi3.png`, 22 -> `wbowi4.png`, 23 -> `wbomb3.png`. Ids 7 and 8 asked nothing (already asked at login); all other ids 0..39 asked nothing. So the table above (7, 12, 20, 21, 22, 23) is all that is known; "bow = 7, bomb = 8" from the server source is only half confirmed (7 yes, 8 not isolated).

B. Bombs, arrows, baddies: who owns the damage (Part 2, fresh runs)

- No hit packet is sent. Across every bomb and arrow trial (leftover logs plus new runs, >100 hits) the real client sent no PLI 26 (hurt player), 46 (hit objects) or 41; the only messages after a hit were the props message (CURPOWER + `hurt` GANI) and the knockback walk. The victim's own client judges the damage when a server-sent bomb (PLO 11) explodes near it or a server-sent arrow (PLO 19) reaches it, whoever owns it. A client's own bomb hurts the owner too (c2's own bomb: c2 lost half a heart, knocked 5 tiles away; each trial 1 half heart).
- Damage seen: 1 half heart (CUR 6->5) with a 5-tile knockback away from the bomb/arrow (victim centre minus the blast point), in every deterministic trial; a few runs showed 4 (a full heart) without a rule: OPEN (maybe a second overlapping hit).
- Bomb zone with a 7.5 s wait (explosion done): victim at (30,30.5), bomb x=30: hits at y offsets +1, +3, -3 (bomb y 31.5, 33.5, 27.5); NO hit at 0, +2, -2 (30.5, 32.5, 28.5). With bomb y = victim y+1: hits at x offsets +4, +5, -4. Row offset 0: nothing at x offset 0, +1, -1. This is not a plain radius; it looks like a pattern keyed on the bomb's half-tile y (the first, 4.5 s-wait sweep had hits at every .0/.5 offset except the integers 0, +-2): OPEN, needs the 2.05/2.22 explosion routine, not guessed here.
- Arrows: an injected PLO 19 arrow (owner 3) hurt only when its last byte was 0x23 or more; bytes 0x20 to 0x22 did nothing; the real arrow c2 fires carries 0x21 and never hurt c1: OPEN (probably a flight/lifetime field).
- Baddies: not tested (the lab server has no baddies). OPEN.
- What Seihai should do: nothing changes for bombs/arrows yet; documentation/damage-respawn.md can say "judged by the victim, half heart, 5-tile knockback away from the blast; blast shape open".

Seihai changes that came out of this

- Chests open only facing up: squirelworks/seihai#73.
- Login asks the images of default weapon ids 7, 12, 20-23: squirelworks/seihai#74.
- Packet 40 (range, cap, NPC refusal, zero vector), CURPOWER clamping and the respawn hearts by AP already matched Seihai (#55); no change.

Method (no traces kept here)

Real 2.22 client under Wine in a private network namespace against the lab GServer and list server; server-to-client messages injected by a re-encoding proxy; client messages read from the proxy's decoded log. Hearts also read from screenshots. No public server, no credentials.

`client-internals/feature-specs/03-inventory.md`

03 Inventory

Snapshots read: Graal_analysis `8da6800`, Seihai `06aacfd`, GServer-v2 (`TLevelItem.cpp`, `TPlayerProps.cpp`, `TPlayer.cpp`).

What it is

The player's counts and gear are **player props sent by the client**. The server clips and stores them. Items on the floor are `PLO_ITEMADD/ITEMDEL`; chests are `PLO_LEVELCHEST`.

Item index table (0-24) [srv], used by ITEMADD, ITEMTAKE, chests, DEFAULTWEAPON, WEAPONADD

0 greenrupee (+1 gralat), 1 bluerupee (+5), 2 redrupee (+30), 3 bombs (+5, max 99), 4 darts (+5 arrows, max 99), 5 heart (+1 heart), 6 glove1 (glove power at least 2), 7 bow, 8 bomb, 9 shield (power 1), 10 sword (power at least 1), 11 fullheart (+1 max hearts, hard cap 20, and heals fully), 12 superbomb, 13 battleaxe (sword at least 2), 14 goldensword (sword 4), 15 mirrorshield (2), 16 glove2 (3), 17 lizardshield (3), 18 lizardsword (sword at least 3), 19 goldrupee (+100), 20 fireball, 21 fireblast, 22 nukeshot, 23 joltbomb, 24 spinattack (sets the has-spin status bit). Items 7-8 and 12 and 20-23 become weapons (`addWeapon`). Shields and swords never downgrade. Rupees cap at 9,999,999.

Props involved (self stream, PLPROP ids) [srv]

1 MAXPOWER (hearts), 2 CURPOWER (half-hearts), 3 RUPEESCOUNT (i3), 4 ARROWSCOUNT, 5 BOMBSCOUNT, 6 GLOVEPOWER, 7 BOMBPOWER, 8 SWORDPOWER (`c(power+30)` + image, or a plain power 0-4 for `swordN.png`), 9 SHIELDPOWER (`c(power+10)` + image, or plain 0-3), 18 STATUS bits (`0x40` has spin). Server clipping when the client sets them (`setProps`): arrows and bombs 0..99; rupees at most 9,999,999; sword and shield power clipped to `swordlimit`/`shieldlimit` (3 in the lab); bomb power 0..3; glove max 3; **CURPOWER cannot rise for players under 40 AP**. With V8NPCSERVER some of these become server-only; the lab server is built without it.

Item pickup and chests [srv][cap]

- Floor item pickup: the client sends `PLI_ITEMDEL` (13) `c x*2, c y*2`. The server relays `PLO_ITEMDEL` (23) to everyone else and removes it from the level. It does **not** give the item on 13. `PLI_ITEMTAKE` (32) does the same removal and also applies the item to the player's props (via the table above) and echoes the prop change to the player and the level. 2.05 used 13 in the capture (client then edits its own props). **[cap]**
- Dropping an item: `PLI_ITEMADD` (12) `c x*2, c y*2, c item`; the server relays `PLO_ITEMADD` (22), adding a player id for dropped swords and shields.
- Chest open: `PLI_OPENCHEST` (20) `c x, c y` (whole tiles). If the account has not opened it (`chestList` in the save), the server applies the chest item with the same table, answers `PLO_LEVELCHEST` (4) `c 1, c x,`

`c y`, and records the chest. Level entry sends one LEVELCHEST per chest (`c opened, c x, c y` + `c item, c signindex` if closed). The account's opened-chest list persists (see 05).
- Drops: bushes/vases with `bushitems`, baddies with `baddyitems`, and deaths (`dropitemsdead`, `mindeathgralats`..`maxdeathgralats`, `dropItemsOnDeath`) create ITEMADD packets from the server.
- Weapon list display is in 02.

Seihai today

- `send_open_chest` exists; `main.rs` sends it on A/Space near a chest and takes the reply (`opened_chests`) into the session. The item itself is not applied: the chest item index is printed to stderr only.
- HUD draws `player_health` and `rupees` (`render/tiles.rs draw_hud`). Bombs, arrows, sword, shield, glove, MAXPOWER, AP, status are not displayed or sent.
- ITEMADD/ITEMDEL (22, 23) are unhandled. Own-prop sending is limited to position/sprite/chat.
- Offline mode has its own item model (`offline/mod.rs give_item`) that is not the GServer table.

Steps for Claude

1. Add an `Inventory` struct to `SessionState` filled from the self prop stream: maxpower, curpower, rupees, arrows, bombs, glove, bombpower, sword (power+image), shield (power+image), status bits. Fix decode for both prop forms (see `protocol/props.rs`).
2. Parse PLO 22/23 into a per-level list of floor items (x,y,item); parse PLO 4 chests; draw both.
3. Pickup: when the player's feet overlap a floor item, send `PLI_ITEMTAKE` (32) if the version profile has it (2.05 lists 32), and let the server echo the new props; fall back to 13 plus local prop edits if a server ignores 32.
4. Chest: after the server's LEVELCHEST (opened=1), show the item name from the table and update the inventory locally only if no echo arrives within one tick (the lab echoes via `setProps`).
5. HUD: hearts (CURPOWER/2 of MAXPOWER), rupees, bombs, arrows, selected weapon.
6. Implement an item key (`Q` in the original) to select among default weapons and named weapons; sending a use event is 02's job.
7. Add a unit test per table row (item index to prop change) and a replay test using the lab chest/bush captures.

How to test

- Lab, one Seihai: log in, walk to a bush, bomb it, take the drop, and verify the prop echo. Chest in `onlinestartlocal.nw` if present; otherwise use the RC-free approach of adding a chest to the lab level file (Seihai supports `.nw` reading, the editor in `tools/level-editor` can place one).
- Check server saved state afterwards in the lab account file (`servers/lab/accounts/<name>.txt`): RUPEES, BOMBS, ARROWS, SWORDP, CHEST lines.

NEEDS

nothing blocking. Optional: "the 2.05 item/weapon list window (Q key) layout" for a faithful HUD; not needed for interoperability.

client-internals/id24-205.md

Id 24 in 2.05: the same weapon-image list

In 2.05, client id 24 already carries the weapon-script image list that the 2.17.1, 2.22 and 2.31 clients send (`decomp-findings-2026-10-25-g/01-id24-image-list.md`). It is not an emote, and it has no other meaning. Read from the 2.05 program.

What 2.05 sends

- **The images:** the images that the selected weapon's script shows, at indexes 0 to 199. A single image set aside by the client replaces that list when present.
- **Changes only:** the list is compared with the one sent last.
  - An image that is new or has changed is sent as a record.
  - An image that is gone is sent as its index byte followed by an empty name.
- **Record layout:** a record starts with 0x2a plus the image's index. Tag bytes follow (0x20 plus the tag), as in the later builds:
  - tag 0 is the name;
  - tags 1 and 2 are x and y;
  - tag 3 is the layer, sent only when it is not 1;
  - tag 4 is the part, sent only when its width and height are above 0;
  - tag 5 is the colours, sent only when any is not 1.0;
  - tag 6 is the zoom, sent only when it is not 1.0.
- **Full list:** the full list starts with the marker 0x29 (GCHAR 9), as later. It goes out:
  - when nothing has been sent yet and there are images;
  - and when a countdown of 60 runs out. That is every 61st call, against 101 in the later builds.
- **Copies:** the text always goes out as UDP id 15. The TCP copy (id 24) goes out when UDP is off for the client, or when some player in the level is not reached by UDP. This is the same test as 2.17.1 and 2.22.
- **Empty text:** nothing is sent.

Correction

Earlier notes said 2.05 and earlier use "a four-field form (index, x, y, name)" for this id. That is wrong for 2.05: it uses the tagged form above, with tags 0 to 6. Tags 7 and 8 of the later builds are not sent by 2.05.

Confirmed, inferred, not tested

- Confirmed from the program: the source of the images, the diff, the record start, the tags and when each is sent, the marker, the countdown, and the UDP/TCP split.
- Inferred: that the countdown runs on the same periodic handler as in the later builds, which would give about 3 s.
- Not tested on a wire. Builds before 2.05 were not read.

client-internals/rendering-observations.md

Rendering observations

Date: 2026-10-14

Derived from observed client behavior and public DirectX notes. No decompile material.

Frame loop

The client runs an idle handler that gates on a 50 ms timer, then a frame function that handles clock skew, network traffic, and draw. A stall is one frame; there is no catch-up burst. The frame function ignores the step count that the pacer may supply.

Draw path

The frame function calls a draw routine that uses the currently selected tile sheet. The device object exposes:

- viewport
- material
- projection transform
- ambient render state (set to white)

Supported image suffixes observed in the drop path are `.txt`, `.gif`, `.png`, and `.mng`. The default sheet is `pics1.png`.

Tile sheet and blit

The active sheet is selected from the loaded definition list when the level is entered. An offline session that already draws an ASCII or PPM view of the board can replace that view with a blit of the selected sheet once:

1. The sheet is chosen (default `pics1.png`, type 0).
2. The type-0 allow lists are applied so only walkable tiles are treated as open.
3. The stamp vocabulary is used so a named stamp expands to the correct tile ids.

A single stamp blit (for example the 2×2 Bush) is enough to prove the path. After that, the full board can be blitted from the same sheet.

Offline implication

The next useful increment remains small:

1. Blit one stamp from `pics1` using the sheet the original client would have selected.
2. Wire `playertouchsme` so that a touch actually hides the NPC or warps the player.

After those two changes the offline session starts to look like a level instead of a parser. Both steps use data and rules already documented in `formats/tile-walk.md` and `npc-library/gs1-commands.md`.

`client-internals/status-display.md`

Status display (HUD)

Date: 2026-10-28

- **Code:** read from the 2.05 rev 2 client's status routine, with two constants checked in its executable: the heart-row table and the small-heart stagger. **[2.05]**
- **Sheets and picture:** the shipped sheets, and a 2.22 lab screenshot. **[cap]**

- **What these notes leave out:** addresses, routine names and decompiler text.

Sheets

- Every build from 1.41 on ships a 224×145 `state.png`.
- The 2.22 sheet matches the 2.05 rev 2 and 2.1 test 3 sheets pixel for pixel, except for one 27×14 patch at (190, 60), which the status routine does not use.
- 1.41's sheet has three blank buttons side by side at x 158, 180 and 202.
- The 2.0-era NewMain sheets have one button at x 202, which the routine draws in all three slots.

Layout

All positions are measured from the top-left of the game view. The routine uses a base point of (15, 15).

Part	`showstats` bit	Sheet rect (x, y, w, h)	Screen position
Rupee, bomb and arrow icons	0x02	(80, 33, 120, 16), one strip	(274, 30)
Rupee count	0x04	digit d = (14d, 81, 14, 16)	1–7 digits, no leading zeros, 12 px apart, centred on x 280, y 48
Bomb count	0x08	same digits	two digits at x 326 and 338, y 48
Arrow count	0x10	same digits	two digits at x 374 and 386, y 48
Hearts	0x20	big (40·half, 0, 40, 44); small (80 + 31·half, 0, 31, 33)	see below
Buttons	0x01	(202, 0, 22, 33)	x 15, 80 and 145, y 30
Key names	0x01	text	on each button: 26 pt Helvetica, white, with a black copy at +1, +1; smaller sizes for two-letter and longer key names. Key 9 (item list) is drawn small at (147, 27)
Glove icon	0x01	sprite 145 for power 2; sprite 325 drawn on top for power 3	(40, 29); power printed at (60, 31) when it is 2 or more
Sword icon	0x01	sprite 17, taken from the player's sword image	(105, 31); sword power printed at (125, 31)
Selected weapon	0x01	the weapon's whole image	x 170, vertically centred on y 45
Alignment bar	0x40	frame (0, 97, 130, 22); fill (0, 119, AP, 10)	(15, 65); fill at +22, +6
Magic bar	0x80	frame (0, 49, 130, 22); fill (0, 71, MP, 10)	(15, 90); drawn only with weapons on and MP > 0

Bits 0x100 and 0x200 belong to the player icons and to the stacked weapon image list, which are drawn by the same routine but are not part of the status display.

- **Sprite rects.** Sprites 17, 145 and 325 come from the sprite table in `arrays.dat` (array 24). The 2.22 file gives:
  - 17: (94, 36, 28, 28), from the sword image
  - 145: (28, 212, 28, 32), from `sprites.png`
  - 325: (0, 212, 28, 32), from `sprites.png`
- **Weapons flag.** The same flag gates the S and D keys. With it off, only the first button, its key name and the alignment bar are drawn.

Hearts

- **Drawing.** One heart is drawn per container while power remains; empty containers are not drawn. Power is a value in hearts: heart *i* is full when `power − 1 ≥ i` and half when `i < power`.
- **Big hearts.** Used while power is 4 hearts or less. They start at (W − 198, 24), 45 px apart, where W is the view width.
- **Small hearts.** Used above 4 hearts, 35 px apart. New rows start at hearts 5, 11, 16, 22 and 27. Row *k + 1* is 20(k + 1) px lower and shifted `round(17.5(k + 1)) − 35` px sideways. The whole block moves 10 px down while power is below 6 hearts.

Not read

- The 2.22 and 2.31 status routines. Their sheets and the 2.22 picture match 2.05.
- The routine of any build before 2.05, including how 1.41 uses its three button cells.
- The text drawing that prints the glove and sword power: its font, colour and anchor.

The selected weapon (2.05 rev 2)

The image beside the D button belongs to the selected entry of the player's weapon list.

- **One list.** Server weapons (`PLO_NPCWEAPONADD`) and built-in weapons (`PLO_DEFAULTWEAPON`) share one list, in arrival order. A built-in weapon is skipped when the list already has its name.
- **Adding.** Every add selects the new last entry.
- **Selecting index *i*.** The client takes the first entry from *i* on, wrapping round, whose name is not empty and does not start with `-`. If every entry is hidden, *i* stays selected.
- **Deleting.** The selection keeps its index if that index still exists. Otherwise it moves to the new last entry.

Built-in weapons

The client keeps a 25-entry item table. Only the entries with a weapon script become weapons. The weapon's name is the item name with its first letter in upper case.

id	item	image (2.05)
7	bow	`wbowi1.gif`
8	bomb	`wbomb1.gif`
12	superbomb	`wbomb2.gif`
20	fireball	`wbowi2.gif`
21	fireblast	`wbowi3.gif`
22	nukeshot	`wbowi4.gif`
23	joltbomb	`wbomb3.gif`

- **The other ids:** greenrupee, bluerupee, redrupee, bombs, darts, heart, glove1, shield, sword, fullheart, battleaxe, goldensword, mirrorshield, glove2, lizardshield, lizardsword, goldrupee and spinattack. They are pickups, not weapons.
- **2.22:** in the lab the client asked for the same names as `.png`.
- **Id 8 and `wbomb1`:** the 2.05 table ties them together. The lab capture could not tie them, because 2.22 asked for `wbomb1.png` whether or not id 8 was sent.

`client-internals/body-colours-and-attributes.md`

Body colours, looks commands and GANI attributes (2.05)

How 2.05 colours a player's body, what the looks commands change, and how a GANI's attribute sprites (the hat) pick their picture. Derived from the 2.05 client unless marked otherwise.

Body colours

Fact	Status
A body picture is a palette image. Before it is drawn, palette entry 5 takes the skin colour, 6 the coat, 4 the sleeves, 3 the shoes and 1 the belt. Entry 0 always takes colour 18 (black, the outline)	derived
The five colours are prop 13 (COLORS), in the order skin, coat, sleeves, shoes, belt. This settles the order `protocol/player-props.md` marks as inferred	derived
The colour list, by index: 0 white 255,255,255; 1 yellow 255,255,0; 2 orange 255,173,107; 3 pink 255,132,132; 4 red 255,0,0; 5 darkred 206,24,41; 6 lightgreen 132,255,132; 7 green 0,255,0; 8 darkgreen 0,198,0; 9 lightblue 132,132,255; 10 blue 0,0,255; 11 darkblue 0,0,198; 12 brown 132,0,0; 13 cynober 0,255,255; 14 purple 255,0,255; 15 darkpurple 132,0,132; 16 lightgray 206,206,206; 17 gray 132,132,132; 18 black 0,0,8; 19 transparent 0,0,0	derived
A new player is orange, darkred, black, darkred, black (2, 5, 18, 5, 18), with `body.png` and `head0.gif`	derived
The stock bodies follow that palette layout, and entry 2 is the clear colour	checked on the 2.22 `body.png` and `body13.png`
Graal Reborn's GServer gives a new account 2, 0, 10, 4, 18 instead	read from GServer-v2 beta4 source

Looks commands

Fact	Status
`sethead`, `setbody`, `setskincolor`, `setcoatcolor`, `setsleevecolor`, `setshoecolor` and `setbeltcolor` change the local player, whichever NPC or weapon runs them. The NPC is not changed; an NPC's own look is `setcharprop`	derived
A colour command takes a colour name from the list above or a number, rounded, into its slot	derived
`sethead`, `setbody`, `setskin`, `setcoat`, `setsleeves`, `setshoes` and `setbelt` typed in chat are run by the GServer, which sends the new props back. They are not client commands	read from GServer-v2 beta4 source

GANI attributes

Fact	Status
A GANI sprite whose picture is `ATTR1` … `ATTR5` draws the player's attribute 1 to 5	derived
Attributes 1 to 5 are props 37 to 41 (GATTRIB1 to GATTRIB5). 2.05 reads them for other players as well as its own	derived
A player with no attribute of their own draws the GANI's `DEFAULTATTR1` … `DEFAULTATTR5`. The stock `idle.gani` names `hat0.png` as attribute 1, the hat	checked on the 2.22 GANIs

Fact	Status
`setplayerprop #P1,<image>` … `#P5` set attributes 1 to 5; `#3`, `#8` and `#C0` … `#C5` set head, body and colours the same way (`npc-library/gs1-command-reference-205.md`). What a sixth colour slot `#C5` is for is not known	derived

Not checked

- Which folder 2.05 looks in for an attribute picture. The stock hats are in `hats/`, so the hat is inferred to be looked for there.
- What 2.05 draws for an attribute set to an empty name.
- None of this has been checked against a running 2.05 client.

`client-internals/chat-text.md`

Chat text and nicknames over players

How the classic Windows clients draw the chat line (`PLPROP_CURCHAT`, prop 12) over a player, and how long it stays. Read from the client binaries (1st Graal to 2.31) and checked on the real 2.05 rev 2 client under Wine on 2026-10-30.

There is no bubble. Every build draws the text straight onto the game view with the Windows canvas (`TCanvas.TextOut`), in a bold TrueType font, with no box behind it.



real 2.05 rev 2 (top) and Seihai (bottom), "hello world Wg"

Font per build

The font is a global pair, name and `Font.Height`, set before each draw. From 1.31 on, `font=` and `fontsize=` in the client config replace the defaults (see `graalconfig-reference.md`). The style is always bold (`fsBold`).

Builds	Face asked for	Height	Shadow colour	Source
1st Graal, 4th, 9th, 15th, 1.1.9, 1.21	`Dialog`	28	always blue	literals at each draw call
1.31, 1.38	`Dialog`	28	contrast rule (below)	font globals, `font=`/`fontsize=`
1.39, 1.41	`Helvetica`	24	contrast rule	font globals
NewMain 1.42, 2.00 to 2.05 rev 2, 2.17, 2.22, 2.31	`Tempus Sans ITC`	24	contrast rule	font globals; `tempsitc.ttf` is installed from the client folder with `AddFontResourceA`

Notes: - **Height.** The value is written to `Font.Height`. It is positive, so it is the cell height in pixels (ascent plus descent), not the point size. For Tempus Sans ITC (2048 units per em, Windows ascent 1974, descent 698) a cell of 24 px is an em of about 18.4 px. The live capture matches this size: "hello world Wg" is 130 px wide and 20 px from the top of the `h` to the bottom of the `g` in both the real client and a rasteriser set to cell height 24. - **No such font.** Windows has no font named `Dialog`. The GDI font mapper then picks a sans serif by charset and pitch, which is Arial on a normal install. `Helvetica` is mapped to Arial by the standard `FontSubstitutes` entry. Neither of these was checked on a real Windows machine. - **Bold.** Tempus Sans ITC ships only a regular face, so Windows makes the bold itself: each glyph is smeared one pixel to the right and each advance grows by one pixel. - **6.x.** The v6 thread reports the same default (Tempus Sans ITC, size 24) and the same colour, shadow, anchor and wrap rules in the Linux 6.03 renderer. See `protocol/v6x-clients.md` when it lands.

Colour and shadow

- **Text colour.** White (`0xFFFFFF`) for chat.
- **Two passes.** The shadow is drawn first at (+1, +1), then the text at (0, 0).
- **1.21 and earlier.** The shadow is always pure blue, RGB (0, 0, 255).
- **1.31 and later.** The shadow colour is worked out from the text colour. If the red or the green of the text is above `0xE0`, the shadow's red and green are 0. Otherwise they are `0xFF`. The shadow's blue is always the text's own blue. For white text that gives (0, 0, 255), the same blue as before. The same routine draws other text over sprites, such as the nickname under a player and number pop-ups, so darker text gets a light yellow shadow.

Position

The routine takes a point in tiles and centres the text on it horizontally.

- **x.** The player's x + 1.5 tiles, the middle of the 48 px player box.
- **y.** The top of the text cell is the player's y − 1 tile, so the text sits one tile above the top of the player box. When the player carries something, it is y − 2 tiles. The carry part comes from a field of the player object that is non-zero while something is held. It was read from the code, not checked live.
- **Nickname.** The name is drawn by the same routine at y + 2.5 tiles (see below). In the capture the chat glyphs were 56 px (3.5 tiles) above the nickname glyphs, which matches.

The view is clipped to the level view rectangle (`TextRect`).

Nickname (1.31 to 2.31)

The nickname is drawn by the same routine as chat, under the player. Wrapping and the shadow rule are the same as for chat.

- **Position.** Centred on x + 1.5 tiles. The top of the text cell is at y + 2.5 tiles, 40 px below the top of the player box.
- **Colour.** The colour depends on the player's AP (alignment, prop 32). Each step below is `floor(n × k / d)`, computed with a 0.0001 bias before truncating.

AP	Colour (R, G, B)
0	(64, 0, 0)
1 to 26	(128 + (ap−1)·127/25, 64 − (ap−1)·64/25, same as G): dark red to red
27 to 49	(255, (ap−26)·255/24, same as G): red to pale pink
50 to 55	white
56 to 60	(255 − (ap−55)·127/5, 255, same as R): white to light green
61 to 70	(128 − (ap−61)·128/10, 255, 128 + (ap−61)·127/10): light green to cyan
71 to 99	(0, 255 − (ap−70)·255/29, 255): cyan to blue
100	(224, 196, 0), gold

The same ramp's byte sequence is present in 1.31, 1.38, 1.39, 1.41, NewMain 1.42, 2.00, 2.05, 2.17, 2.22 and 2.31. 1.21 has no AP colours. The v6 thread reports an AP ramp for 6.x too.

Checked live on 2.05 rev 2 by sending prop 32 for another player. The drawn colours for AP 0, 20, 40, 58, 65, 85 and 100 were exactly (64,0,0), (224,16,16), (255,148,148), (179,255,179), (77,255,178), (0,124,255) and (224,196,0). AP 50, which is also the value before any AP is sent, was white.

- **When it shows.**
  - *Other players.* Always.
  - *Your own.* Only for 60 frames (3 s) at a time. A per-player counter is set to 60 when:
    - the player is set up, which is why it shows at login;
    - the player's AP changes, from a packet or a script;
    - you type exactly `showname`. That line is not sent to the server.

    The counter counts down in the own player's tick, and the name is drawn while it is 0 or more. Other players' counters never count down.

  Live: after login the own name went away. A typed `showname` brought it back for 3.0 s: it was up from the first frame, still up at 3.11 s and gone at 3.32 s. Nothing was sent.

- **Length.** It is cut to `nicknamelimit` characters only when `limitnicknames` is on. That option is off by default, and the limit is 50.

Wrapping (1.31 and later)

When the text is wider than the view (640 px in 2.05), it is split at spaces:

- Words are added to the line while it still fits.
- A word that does not fit starts the next line.
- A word that does not fit on a line of its own becomes a line by itself, unbroken.

The line height is `TextHeight(' ')`, which is the cell height. Lines stack upward: the first line is drawn (lines − 1) × line height above the anchor, so the last line stays on the anchor.

Each line is centred on its own. If it would run past the right edge it is moved left until it fits, and then moved right if it would start left of 0.

1.21 and earlier draw a single centred line, clipped to the view, with no wrapping.

The 6.x renderer wraps only text of 11 or more characters. It breaks after any of `.,;+-?!` (v6 thread). The classic builds break at spaces only.

Special texts (2.05)

- **`showap`.** A chat of exactly `showap` shows the player's AP number instead of the text.
- **Other displays.** The same slot above the head is used by other things. While a per-player counter runs, it shows a pair of numbers joined by `/`. In some sprite states it shows an item image or a number from what the player carries. In those cases the chat is not drawn. These branches were read but not named.

How long it stays (live, 2.05 rev 2, 2026-10-30)

One frame is 50 ms. The live timings below confirm the 60-frame = 3 s reading.

- **Your own line.** It stays up as long as you stand still: it was still up after more than 7 minutes. The first direction key starts the countdown, and the line went 3.0 s later (between 2.92 s and 3.11 s in screenshots taken every 0.2 s). At that moment the client sent an empty CURCHAT.
- **Another player's line.** It shows for at least 3 s from when it arrives, and until that player's empty CURCHAT arrives. Three runs against the lab stub server:
  - Empty line sent 20 s or more after the text: the text was gone in the next frame (under 0.15 s).
  - Text, then empty after 2.0 s: gone about 1 s after the empty, 3.0 s after the text.
  - Text, then empty after 0.54 s: gone between 3.06 s and 3.21 s after the text.
  - With no empty line, it stays: it was still up after 10 s.

This corrects `seihai/documentation/chat-log.md`. That page said an empty CURCHAT starts a fresh 60 frames. In fact it only waits out whatever is left of the 60 frames that began when the text arrived.

Length

When the line is sent, a chat longer than 256 characters is cut to its first 253 characters plus `...`. The receiving client draws whatever arrives.

Method

- **Static.** Every client's font globals were read: the `Tempus Sans ITC`, `Helvetica` or `Dialog` string followed by the height dword. The 2.05 rev 2 text routine and `TPlayer` draw were disassembled. The shadow rule's byte sequence is identical in 1.31, 1.38, 1.39, 1.41, NewMain 1.42, 2.00 to 2.05, 2.17, 2.22 and 2.31.
- **Live.** The 2.05 rev 2 client ran under Wine in the lab (`graal-decompile/live-trace/SETUP.md`), with chat injected from the stub server as `PLO_OTHERPLPROPS` prop 12. The screenshot comparison above puts Seihai's renderer next to it, using the `tempsitc.ttf` shipped with the 2.22 install. Wine anti-aliases the text. A Windows 98 machine without font smoothing would draw the same glyphs with hard edges.

`client-internals/sign-panel.md`

The sign panel and NPC drawing (2.05)

How the 2.05 client draws a sign while the player reads it, the keys that turn its pages, and what it draws for an NPC whose image is missing. Read from the 2.05 program and checked against the `letters.png` sheets of 1.39 and 2.22. Seihai follows this in `documentation/signs.md`.

The sheet

`letters.png` is 256×224: 16 glyphs a row, each 16 wide and 32 tall, so glyph `g` is at (`g % 16 × 16`, `g / 16 × 32`). It carries the font, the button pictures and the pieces of the scroll frame. The client loads it once, at start.

Glyph order: a character's glyph is its place in

```
ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789!?-.,#>()#####"####':/~&### <####;
```

(then a newline). The `#` places are glyphs with no plain character. Each glyph has its own advance, the client's table of half widths doubled (I is 6 pixels, space 8, most letters 12).

glyphs	use
96, 97, 98, 99	frame corners: top-left, top-right, bottom-left, bottom-right
100, 101	top and bottom edge
102, 103	left and right edge (32 pixels tall)
91 to 94	the A, B, X and Y buttons
77 to 80	the up, down, left and right pictures
71 to 75, 87, 88, 67	the other button pictures

The pixels at (252..255, 223) are colours, not art. The client fills the inside of the frame with (254, 223) and, 8 pixels further in, (255, 223), and skips a fill whose colour is the sheet's transparent one. On the 2.22 sheet both are the same cream; on 1.39 the inner one is white.

The panel

- 24 cells wide and 9 tall (16-pixel cells), centred across the view.
- On a 640-wide view it sits at y 256, or at y 128 when the player is more than 14 tiles down the view. At 800: 18 tiles, y 320 or 160. At 1024: 22 tiles, y 448 or 192. Which two values the client compares is inferred from the code's shape (the player's row against the view's top); the thresholds and positions are as read.
- The frame: the top and bottom edges every 16 pixels, the sides every 32 pixels, the corners over them, then the two fills from (x + 16, y + 24) to the right and bottom pieces.
- Text: three lines a page, from (x + 16, y + 16), 32 pixels apart. The sign's text is split at newlines.
- In a line, `#` and a letter draws button pictures: `#A`, `#B`, `#X`, `#Y`, `#u`, `#d`, `#l`, `#r`, `#h`, `#x`, `#y`, `#z`, `#4`, `#.` . `#` and any other letter draws glyph 86 and moves 16 pixels. `#i` and a character puts up an image from the level's list.
- A character outside the glyph order is drawn by the client in its system font, in the colour at (253, 223).

Keys while reading

Reading a sign is its own input state: walking and the A / S / D handler are only reached outside it, so the player does not walk, swing, lift or use an item while a sign is open. Switch-to-chat still works, and while the chat bar has focus the sign keys are skipped. **[2.05]**

- The map key ends reading at once.
- The first direction key (Up) turns back three lines, when not on the first page.
- Every other game key (Left, Down, Right, D, S, A, item list, pause) turns to the next three lines, unless the page on show is not full (the sign has fewer lines than the page's first line plus three); then reading ends. A sign of exactly three lines therefore shows an empty second page before it closes, as in the client.
- A key counts only when no game key was down on the tick before, so a held key does not repeat and the key that walked into the sign does nothing until it is let go.
- The client plays a sound when the page turns and runs `playerendsreading` when reading ends.

(An earlier version of this note said any other key ends reading; only the map key does.)

NPCs with no image

- In game, an NPC with no image loaded draws nothing of its own, only its text if it has any.
- An NPC with a GANI draws the GANI, and a character NPC is drawn the way players are.
- The client's placeholder picture for an NPC with no image appears only in the level editor's view.

Confirmed, inferred, not tested

- Confirmed from the program: the glyph grid, glyph order and advances, the frame pieces and fill colours, the panel size and line layout, the # codes, the paging rule, and the NPC draw rules above.
- Inferred: which two values the panel's height test compares, and that key 0 is Up.
- Not tested against a real client screenshot.

`client-internals/feature-specs/09-options.md`

09 In-game options

Snapshots read: Graal_analysis `8da6800` (`client-internals/graalconfig-reference.md`, `config-and-input.md`), Seihai `06aacfd` (`settings.rs`, `bin/main.rs`).

What it is

The 2.05 client has a **More Options** dialog, a **Player List Options** dialog and a **Controls** frame. All of them read and write one text file, `graalconfig.txt` (`key=value`, case-insensitive keys, rewritten in full with every key in fixed order whenever either dialog closes). Nothing goes over the network. Registry values hold the nickname, account list and (encrypted) passwords.

Parse rules (matter for a compatible reader) [2.05]

- Booleans are true only if the lower-cased value is exactly `true`; a **missing** boolean reads as false.
- Integers: parse error keeps the default. Strings: used only if non-empty.
- Key binding: a key already bound earlier in the file becomes unbound; binding a key held elsewhere swaps it.

Keys (file order, default)

`waitvertical` false, `musicon` true, `usewinamp` false, `soundfxon` true, `reversestereo` false, `saveimages` true, `dontsavepms` false, `compabilitydraw` false (spelling as in the client), `nodecompressionerror` false, `head` (default `Fung Si Yun`), `joystickon` true, `nodirectinput` false, `joyselected`, `keyset_<action>_joystickbutton`, `keyset_<action>_key0..2`, `language` (English), `font` (Tempus Sans ITC), `fontsize` 24, `fixedport` false, `fixedudpport` 14899, `noudp` false, `nomassmessages` false, `nohtmlinpms` false, `limitnicknames` false, `nicknamelimit` 50, `dontconnectlevels` false, `notoalls` false, `nohtmlimages` true, `sortplayersbyname` false, `dontloadlistheads` false, `dontsavepasswords` false, `dontusedirectx7` false, `fullscreenmode` 1024x768. Actions with three key slots: up, left, down, right, item, attack, action, map, switchtochat, itemlist, pause. The per-action default keys for 2.05 are not listed in the notes; the fixed keys of 1.37 were arrows or IJKL, D item, S or Space attack, A action, M map, Q item list, P pause, Tab chat (`versions-133-139.md`). Default joystick buttons are in `graalconfig-reference.md`. Later clients add `midivolume`, `mp3volume`, `radiovolume`, `sfxvolume`, `voicevolume`, and an Audio/Microphone tab (2.17+).

What the options change at run time

`musicon`/`soundfxon`/`reversestereo` (01); `noudp`/`fixedport`/`fixedudpport` (07); `nomassmessages`, `notoalls`, `nohtmlinpms`, `nohtmlimages`, `sortplayersbyname`, `dontloadlistheads` (14, player list); `dontconnectlevels` stops neighbour-level preloading (06); `limitnicknames` truncates displayed nicknames; `saveimages`; `language`.

Seihai today

- `settings.rs` is Seihai's own launcher file (assets path, account, password, version, last server, listserver, offline flag). It is **not** `graalconfig.txt`. It stores the account password in the settings file; note that 2.05 encrypts saved passwords and has `dontsavepasswords`. Decide with the user before keeping plaintext.
- `bin/main.rs` has fixed keys (arrows move, A/Space interact, Enter chat, F2/F3/F4/F5/F9/F12 dev actions, PageUp/Down scroll). No options screen. The launcher `gui.rs` (egui) picks assets and server.

Steps for Claude

1. Add `options.rs`: a struct with the keys above, `load_graalconfig(path)` implementing the parse rules, `save_graalconfig(path)` writing all keys in order. Keep Seihai-specific keys in `settings.rs`.
2. Round-trip test: parse a 2.05-style file with a missing boolean and assert the false-on-missing behaviour; write and re-read.
3. Key map: a table from action to up to three key names; implement the "already bound is unbound" load rule. Replace the hard-coded keys in `main.rs` with lookups.
4. In-game screen: an egui or framebuffer window opened with F1 (original key) listing music, sound, reverse stereo, UDP options, nickname limit, no-toalls, player-list sort. Apply without restart where easy.
5. Wire only options whose feature exists (sound after 01, UDP after 07, gmap after 06). Others are stored and written back unchanged.
6. Optionally read an existing `graalconfig.txt` from the assets folder as defaults.

How to test

- Pure unit tests (file parse/write). No lab needed.
- Compare against a real file written by the 2.22 client in the lab (the notes say 2.22 uses the same keys in the same order).

NEEDS

nothing.

`client-internals/feature-specs/10-right-click-menu.md`

10 Right-click context menu (player list popup, profile)

Snapshots read: Graal_analysis `8da6800` (`versions-133-139.md`, `version-profiles/141-217.toml`, `protocol/chat-pm-rc.md`, `client-architecture-205.md`), Seihai `06aacfd`, GServer-v2 (`TPlayerRequestText.cpp`, `TPlayer.cpp` PM/profile handlers). Folded here from the public-server report: **PLO_STAFFGUILDS 47** (player-list "Staff" section).

What is known [2.05][inf]

- There is no context menu on the wire. It is client UI. The 2.05 client has a `TPlayerList` form (F7) with a popup menu. **[2.05][dec]** The 2026-10-24 decompile gives the item order: **Message (default item), History, Buddy, Ignore, Profile**. The 1.33 build had Message, Ignore, History; Buddy came in 1.35 (`buddyaccounts.txt`); Profile and the Player List Options window came in 1.39. "Right-click profile" is a listed feature in the 2.17 profile.
- **`TPersonAction` is not a player menu.** It is the level editor's "Set Npc's Action" script editor (opened from the NPC editor, see `level-editor-205.md`). There is no per-player action form (trade, duel and so on) in 2.05. An earlier version of this spec listed it as an unexplained 2.05 form; that is corrected.
- Window facts: the list is a stay-on-top form with a multi-select list view; a panel caption reads "--- N online ---"; buttons Profile and Options. Right-click selects the row under the cursor, then sets the Ignore and Buddy check marks to "checked if every selected player is in that list". Double-click or Enter opens the read-PM window if a PM from that player is pending, otherwise the compose window. PM window titles are "Send Message" and "Read Message". The Options window has 5 check boxes; the profile edit window has 10 fields.
- Actions behind the items, with what is known:
  - **Message**: opens the PM box for **all selected rows** (an existing PM window for the same set is reused); sends `PLI_PRIVATEMESSAGE` (28) `s count, s player...` + text. Recipient encoding in 2.05: a two-byte count, then two bytes per player id, **at most 50 recipients**. Done in Seihai (`send_pm`).
  - **Profile**: uses only the **first** selected player and, when online, sends `PLI_PROFILEGET` (80) with the account; the listserver answers `PLO_PROFILE` (75) with 10 length-prefixed fields; fields 1-9 are saved locally. Edit own profile with `PLI_PROFILESET` (81). GServer-v2 does **not** answer profile requests; the listserver does.
  - **Ignore**: local only. Toggles each selected account in `ignoredaccounts.txt` (1.34+: by account name) and saves the buddy list too; ignored players are drawn red in the list (1.34+) and their chat and PMs are dropped by the client. The client holds an auto-reply string "Your messages are ignored."; the condition under which it is sent was not traced.
  - **History**: opens the message history window (not traced further).
  - **Buddy**: local `buddyaccounts.txt` in 2.05, toggled for each selected account; **Ignore and Buddy are mutually exclusive**; see 14 for 2.17+.
- Player list sections: 1.33 had GPs, Guild, Normal, Ignore. 2.14+ lists staff in a section named from `PLO_STAFFGUILDS` (47): a list of guild names that count as staff (GServer `staffguilds` option, sent at login by `sendLogin`).

- Right-click on a player's sprite in the world: **no such handler was found in the 2.05 decompile**. The only player popup is the F7 list's.

Seihai today

- `ui.rs draw_player_list` shows the other players; `nearest()` picks the closest. No menu, no mouse handling beyond the egui launcher. `PLO_STAFFGUILDS` (47) and `PLO_PROFILE` (75) are not handled; `send_pm` exists.

Steps for Claude

1. Add mouse input to the framebuffer loop (minifb gives `get_mouse_pos`, `get_mouse_down`). Right-click inside the F7 list selects a row and opens a small menu drawn with `ui.rs` primitives, in the 2.05 order: Message (default, bold), History (hide until it exists), Buddy, Ignore, Profile. Support multi-select for Message; set the Buddy and Ignore check marks as above and keep them mutually exclusive. Double-click or Enter opens the PM compose box.
2. Message: open the existing chat input in PM mode with the target prefilled.
3. Profile: send `PLI_PROFILEGET` (80) and render the reply when PLO 75 arrives (implement the 10-field parser from `chat-pm-rc.md`; drop field 0).
4. Ignore: keep a set of account names, saved to `ignoredaccounts.txt` next to the settings (one account per line). Filter chat/PM/toall/bubbles for them and tint the row.
5. Buddy: see 14.
6. Store 47 and draw a "Staff" section label for players whose guild matches.
7. Do not add a menu for a world click: 2.05 has none.

How to test

- Headless unit tests on the menu model (selection, ignore filter). Visual check under Xvfb on display :9 only.
- Lab: with a lab account PLI 80 reaches GServer, which sends nothing back (expected); profile display needs a listserver reply, so test with a recorded 10-field PLO 75 frame in `lab_replay`.

NEEDS

Filled by the 2026-10-24 2.05 decompile: item order, check-mark rules, PM recipient encoding, what `TPersonAction` is, and that no world right-click exists (all above).

Still open (none blocks a menu): - The 1.39 popup items (its menu strings are built character by character and were not re-read). - Where the "Your messages are ignored." auto-reply is sent from. - What the History window shows (not traced). - Enable/disable (greyed) rules beyond the check marks, if any.

`client-internals/feature-specs/12-motd-rules.md`

12 Server MOTD and rules

Snapshots read: Graal_analysis `8da6800`, Seihai `06aacfd`, GServer-v2 (`TPlayerLogin.cpp` `sendLoginClient`, `TServer.cpp` message load, `CWordFilter.cpp`, `TPlayerProps.cpp`), the lab server config. Folded here from the public-server report: **PLO_STARTMESSAGE 41, PLO_RPGWINDOW 179**, and the empty `PLO_SERVERTEXT 82`.

Packets (server to client, at the end of login) [srv]

Id	Name	Payload	Versions
41	STARTMESSAGE	the server message as one text string	1.x-2.x; 2.05 has it (client form `TServerMessage`); newer 5/6 clients ignore it
179	RPGWINDOW	quoted, comma-separated lines: `"Welcome to <server name>.","<vendor> <app> programmed by <credits>."`	2.1+ (in the 2.17/2.22 sets)
82	SERVERTEXT	empty at login; the server sends it "to allow serverwarp and some other things". Later it carries replies to `PLI_REQUESTTEXT`/`SENDTEXT` as `weapon,type,option,data` tokens	2.1+
171 / 172	BIGMAP / MINIMAP	see 06	2.1+

`sendLoginClient` order: level data (warp), bigmap, minimap, RPG window, start message, empty server text, flush.

The message text (`TServer`) [srv]

Loaded from `config/servermessage.html`, then every `\r` is removed and every `\n` replaced by a space, so it is **one long line of HTML**. The lab file is the default HTML sample (head, style, a `div` with a welcome and a changelog block). The client is expected to render HTML or at least strip tags.

"Rules" in GServer

`config/rules.txt` is the **word filter**, not a player-facing rules text. Syntax: optional `WARNMESSAGE`, `SHOWWORDSTORC`, then `RULE` ... `RULEEND` blocks with `CHECK` (pm, chat, nick, toall), `MATCH` (`?` wildcard), `PRECISION` (number or percent), `WORDPOSITION` (full, start, part), `ACTION` (log, tellrc, replace, warn, jail, ban) and an optional per-rule `WARNMESSAGE`. `CWordFilter::apply` runs on chat (prop 12), PM (`msgPLI_PRIVATEMESSAGE`), toall, and nickname changes. `replace` swaps matched letters for `*`; `warn` replaces the whole message with the warn text; `log`/`tellrc` go to server log and RC; **`jail` and `ban` are stubs and do nothing**. For chat, if the text was changed the server echoes the new CURCHAT to the sender so the client's own bubble shows the censored text. Real "rules" a player sees come from the server message, the RPG window and NPCs.

Seihai today

- 41, 179 and 82 are counted as unhandled. Chat text arrives and displays; a censored echo from the server is just another prop update.

Steps for Claude

1. Handle PLO 41: store the raw text in `SessionState.start_message`.
2. Small HTML reducer: drop `<style>`, `<script>` and comments blocks, convert `<br>`/`</p>`/`</div>` to newlines, strip other tags, decode a few entities, and wrap to the chat width. Show it in the existing panel

(`ui::draw_panel`) once after login, dismissed with Enter/Esc. Unit test with the lab `servermessage.html`.
3. Handle PLO 179: parse the quoted list and print the lines to the chat log.
4. Handle 82 with empty payload as a no-op marker (`server_text_ready = true`); log non-empty payloads for later use (14).
5. Offer a `--no-motd` flag for scripts and tests.
6. No filter logic is needed in the client.

How to test

- Lab: edit a **copy** of `servermessage.html`, restart the lab gserver, log in with Seihai, and assert `start_message` equals the file with newlines folded. Add a rule in `rules.txt` (`RULE` with `CHECK chat`, `MATCH`, `ACTION replace`), send a chat line, and check the CURCHAT echo contains `*`.
- Replay frames from the public report only as structure checks; the report notes all four public servers sent 41 and 179.

NEEDS

nothing blocking. Optional: "the 2.05 start-message handler" (does it show the HTML in `TServerMessage` immediately or only on request).

`client-internals/feature-specs/13-ban-kick.md`

13 Ban and kick handling

Snapshots read: Graal_analysis `8da6800` (`protocol/gserv-1999.md`, `packet-ids.md`), Seihai `06aacfd` (`protocol/dispatch.rs`, `net/mod.rs`), GServer-v2 (`TPlayer.cpp` login and cleanup, `TPlayerLogin.cpp` `sendLogin`, `TPlayerRC.cpp`, `TServer.cpp` `isIpBanned`, `TServerList.cpp`).

What it is

Every refusal, kick and ban reaches the client as **`PLO_DISCMESSAGE` (16)**: rest = text, `\r` = line break. The server then closes the socket (or the client does). Seihai already parses 16 into `disconnect_reason`.

Exact server behaviour (messages, in the order they can happen) [srv]

`TPlayer::msgPLI_LOGIN` (steps 1-4), then listserver verification (`TServerList`), then `TPlayer::sendLogin` (steps 5-8): 1. Player limit (`maxplayers`): "This server has reached its player limit." 2. IP ban (`TServer::isIpBanned`, wildcard match against `config/ipbans.txt`, skipped for staff with the modify-staff right): "You have been banned from this server." 3. Client version not in `allowedversions.txt` (ranges `a:b` allowed): "Your client version is not allowed on this server." plus a line with the allowed list. 4. Listserver down: "The login server is offline. Try again later." Listserver rejected the login: the listserver's own text (for example wrong password), then disconnect. 5. Account banned (`BANNED 1` in the account file, not for staff): "You have been banned. Reason: <BANREASON>" (newlines become `\r`). 6. `onlystaff`: "This server is currently restricted to staff only." IP not in the account's `IPRANGE`: "Your IP doesn't match one of the allowed IPs for this account." 7. Account already online: if the old session was idle for 30 s or more, the old one gets "Someone else has logged into your account." and is dropped; otherwise the new login gets "Account is already in use." (the check is case-insensitive). 8. No start level: "No level available." While connected: - Idle: `disconnectifnotmoved` with `maxnomovement` seconds: "You have been disconnected due to inactivity." - More

than 5 unknown packet ids (`msgPLI_NULL`), or garbled props: "Disconnected for sending invalid packets." - RC kick (`msgPLI_RC_DISCONNECTPLAYER`): "One of the server administrators, <name>, has disconnected you" plus " for the following reason: <reason>" if given. - RC ban by account (`msgPLI_RC_ACCOUNTSET` and `msgPLI_RC_PLAYERBANSET`): "<name> has banned you. Reason: <reason>" and the account's `BANNED`, `BANREASON` are saved. - RC account reset: "Your account was reset by <name>". - `PLO_UPGRADEURL` (169) can accompany a disconnect to turn Reconnect into an Upgrade button (2.14+). The word filter's `ban` action is not implemented (12).

Seihai today

- `SessionState.disconnect_reason` is filled; the CLI prints it. No classification, no UI, no retry policy. The public-server run saw clean logins only, so no refusal text has been captured from a public server.

Steps for Claude

1. Add `enum DisconnectKind { PlayerLimit, IpBanned, VersionNotAllowed, LoginServerOffline, ListserverRejected(String), AccountBanned(String), StaffOnly, IpMismatch, InUse, KickedByStaff, Inactivity, InvalidPackets, Other }` and `classify(&str) -> DisconnectKind` (match on the phrase starts above, case-insensitive; keep the raw text).
2. Show the reason in the launcher/GUI on exit with the raw text and a hint per kind (for example VersionNotAllowed shows the allowed list; InUse says wait 30 s).
3. Retry policy: never auto-reconnect after a ban, version, staff-only, IP mismatch or invalid packets; allow a manual retry for InUse, player limit, offline login server.
4. Keep the invalid-packet guard from 02/08: assert in tests that no frame Seihai sends has an id outside the profile's `pli` set.
5. Record the reason in the exit code of the CLI (nonzero, distinct per kind) for lab scripts.

How to test

- Lab (private display :9): set `maxplayers = 1` on a **copy** of the lab options and log in twice; put the placeholder account's `BANNED 1` and `BANREASON test` into a copy of the account file; add the loopback address to `config/ipbans.txt` copy; use an unlisted version string; log in the same account from two sessions; stop the listserver stub. Check the exact text and `classify`.
- Unit tests feeding each phrase to `classify`.
- No public server contact.

NEEDS

nothing. (The text of refusals from the real listserver for a banned or unknown account is not in the captures; capture it only if the user approves a public login.)

`client-internals/feature-specs/14-friends-ignore.md`

14 Friends (buddies), ignore, mute, status

Snapshots read: Graal_analysis `8da6800` (`versions-133-139.md`, `version-profiles/*.toml`, `protocol/chat-pm-rc.md`, `graalconfig-reference.md`), Seihai `06aacfd`, GServer-v2 (`TPlayerRequestText.cpp`,

`TPlayerProps.cpp` status message, `TPlayerLogin.cpp`), graal-serverlist (`TServer.cpp` `msgSVI_REQUESTBUDDIES`). Folded here from the public-server report: **PLO_STATUSLIST 180**.

What it is

Four separate features.

Feature	Version	Where it lives
Ignore a player	1.33; by account name from 1.34 (`ignoredaccounts.txt`)	client-local; ignored players shown red and their chat/PMs dropped
Buddy	1.35 local `buddyaccounts.txt`; 2.17+ also through the listserver	local in 2.05; listserver in later builds
Mute	2.17+	`PLI_MUTEPLAYER` (43): `s playerId, c 1/0` (from the playerlist Mute item)
Status (Online/Away/DND/...)	2.14+	`PLO_STATUSLIST` (180) list, `PLPROP_PSTATUSMSG` (53) index

Additional ignore switches: `nomassmessages`/`notoalls` options and the ADDITFLAGS prop bits (`0x01` no mass message, `0x04` no toall, `0x08` voice off) (09).

Server behaviour [srv]

- **Ignore**: none. The server never learns about it.
- **Mute (43)**: GServer-v2 defines `PLI_MUTEPLAYER` in the enum but registers **no handler**. It falls to `msgPLI_NULL`, which counts as an invalid packet: **after more than 5 such packets the server disconnects the client**. A real 2.22 client does send it (see lab findings below), and two sends in a lab session did not disconnect.
- **Buddies (2.17+)**: the client sends `PLI_SENDTEXT` (the generic text packet) with tokens `GraalEngine, lister, verifybuddies | addbuddy | deletebuddy` (+ account list). `msgPLI_SENDTEXT` forwards those three options (not for guest accounts) to the **listserver**; graal-serverlist answers `msgSVI_REQUESTBUDDIES` with a `PLO_SERVERTEXT` (82) whose tokens are `weapon, lister, buddylist, <comma list>` built from a database query. GServer itself stores nothing.
- **Status**: `PLO_STATUSLIST` (180) at login (2.1+/RC): comma list from `playerlisticons` (lab: Online, Away, DND, Eating, Hiding, No PMs, RPing, Sparring, PKing). The client sets its own index with `PLPROP_PSTATUSMSG` (53); the server validates it against the list length and rebroadcasts it to everyone (`PLO_OTHERPLPROPS`).
- **Player list pulls**: `PLI_REQUESTTEXT` types `pmservers`, `pmserverplayers <server>`, `pmunmapserver`, `pmguilds` go to the listserver for cross-server and guild tabs. **PM** (`PLI_PRIVATEMESSAGE` 28) is already implemented.

Lab findings from a real 2.22 client, 2026-10-24 [cap]

- Player-list window sections: Staff, Buddies, Guild, Normal, Ignore. Right-click on a player gives: Message, History, Buddy (check mark when set), Ignore, Mute, Profile. (This is the 2.22 popup; the 2.05 popup is a different, shorter menu, see 10.)
- **Buddy add and delete send nothing** on the game-server or listserver connection. The client writes `buddyaccounts.txt` locally and deletes the file when it becomes empty. The `verifybuddies`, `addbuddy` and `deletebuddy` requests were **not** sent in this lab, so their token layout is still unseen. The assumption above that 2.17+ buddies go through the listserver is not supported by this client in this setup; it may only trigger for accounts the listserver knows (a database-backed listserver) or at some other moment.
- **Ignore** is local too: it writes `ignoredaccounts.txt`, one account per line, and sends no packet.

- **Mute** sends `PLI 43`: `GSHORT playerId` + `GCHAR 1` (mute) or `GCHAR 0` (unmute), one message per click. The list entry does not move on Mute, unlike Buddy and Ignore, which move it.
- Not captured: the listserver-side buddy list reply and any status change.

Seihai today

- PM and toall send/receive (`send_pm`, `send_toall`); chat shows. No ignore filter, no buddy list, no status UI. 180 is unhandled. `send_text` helpers for `PLI_SENDTEXT`/`REQUESTTEXT` do not exist.

Steps for Claude

1. **Ignore (all versions)**: `ignored: HashSet<String>` loaded from `ignoredaccounts.txt` (one account per line, lower-case compare), applied to chat bubbles, `PLO_PRIVATEMESSAGE` (37), toall. Needs the account name prop (34) for each player; show tinted in the F7 list (10).
2. **Buddy (2.05)**: `buddyaccounts.txt` the same way; mark buddies in the list and print a login notice when a buddy appears (`PLO_ADDPLAYER`/`OTHERPLPROPS` with PCONNECTED). Local only.
3. **Buddy (2.17+, lab)**: implement `send_text(weapon, type, option, args)` using the tokenized form from `TPlayerRequestText.cpp`; send `GraalEngine, lister, addbuddy, <acct>`; parse PLO 82 `buddylist`. Behind a flag until captured.
4. **Status**: parse 180 into the list; send `PLPROP_PSTATUSMSG` (53) when the user picks one (Away on a timer is original behaviour, off by default); show others' status icon index in the list.
5. **Mute (2.17+)**: client-local mute set (same filter as ignore). Optionally also send `PLI 43` (`GSHORT playerId`, `GCHAR 1/0`) as the real client does, but GServer-v2 counts it as an invalid packet, so keep it off by default against that server.
6. Tests: ignore filter on a scripted chat stream; buddy file parse; status prop round trip; assert no `PLI 43` is emitted.

How to test

- Unit tests for the filters (no network).
- Lab: two sessions, ignore one account locally and send chat from it (the second Seihai or the Wine client); status: set a status index and check the other client receives prop 53. Buddy via listserver: the lab stub has no buddy table, so extend the stub to answer `addbuddy`/`verifybuddies` with a `buddylist` text, or run graal-serverlist with its database.

NEEDS

the mute payload is now known (see the lab findings). Still open: a capture of the buddy requests (`verifybuddies`, `addbuddy`, `deletebuddy`) and the listserver's `buddylist` reply, which the real client did not send in the 2026-10-24 lab. That needs a database-backed listserver or another trigger. No decompile needed; 2.05 needs nothing (local files).

`client-internals/feature-specs/01-sound.md`

01 Sound and music

Snapshots read: Graal_analysis `8da6800` (main), Seihai `06aacfd` (main, 2026-10-24 03:23 JST). Tags: **[2.05]** read from the 2.05 client, **[srv]** GServer-v2 source, **[cap]** lab capture, **[inf]** inferred.

What it is

There is **no sound packet**. Sound is purely client-side and is triggered by four things: - GS1 commands `play`, `playlooped`, `stopsound`, `stopmidi`, `setmusicvolume`. - GANI `PLAYSOUND name x y` lines (the sound belongs to a frame). - Built-in events: item pickup, bomb, sword, arrow, baddy death, chest, jump, etc. - Config switches `musicon`, `soundfxon`, `reversestereo`, `usewinamp` (see 09).

The only network part is fetching a missing sound: the client sends `PLI_WANTFILE` (23) and the server answers with `PLO_FILE` (102) via `TPlayer::msgPLI_WANTFILE` / `sendFile`. File transfer is already in Seihai.

2.05 built-in effect names (the client plays these without any NPC): arrow, arrowon, axe, bomb, chest, compudead, crush, extra, fire, frog, horse2, item, item2, jump, lift, lift2, nextpage, put, steps, stonemove, sword, swordon, thunder, water (all `.wav`). **[2.05]**

Exact client behaviour (from `client-internals/sound-and-music.md`) [2.05]

1. One entry point takes (file, level offset x/y, x, y). A leading `+` means loop and is stripped.
2. A missing file while online triggers WANTFILE; the sound plays on a later trigger.
3. Listener = player at (x+1.5, y+2). A sound from a neighbouring gmap level is shifted by 64 tiles. Pan = (soundx - listenerx) * 150, clamped to ±10000, negated by `reversestereo`. Volume = attenuation growing with distance on both axes, floored at -10000 (hundredths of a dB).
4. `.wav` goes to the effect path (only if `soundfxon`); anything else goes to the music path (only if `musicon`).
5. At most 8 effects play at once; a 9th is dropped. Effects are cached by file name. Leaving a level stops looped effects.
6. MIDI music loops until `stopmidi` or another track. `setmusicvolume l,r` is 0..1 each.
7. 2.17+ clients use BASS (effects, tracker modules, streams, URL radio). Same GS1 commands plus `musicpos`/`musiclen`. **[2.17][2.22]**

Seihai today

- `assets/gani.rs` parses `PLAYSOUND` into `GaniFrame.sound` (a name only). Nothing plays it.
- No audio dependency in `Cargo.toml` (only flate2, bzip2, minifb, eframe/egui).
- GS1 command table in `scripting/gs1.rs` lists the sound commands by opcode but there is no executor for them.

Steps for Claude

1. Add a `sound` module with a trait `SoundOut { play(name, pan, vol, looped), stop(name), stop_all_loops() }` and a null implementation, so the rest compiles and tests without a device.
2. Write a small WAV reader (PCM 8/16-bit, mono/stereo; the 2.05 files are plain PCM). Cache decoded buffers by lower-cased base name.
3. Implement the mixing rules above: 8-voice cap, pan and volume from listener distance, `+` loop prefix, `reversestereo` flag.
4. Real backend behind a cargo feature (for example `rodio` or `cpal`) so the default build stays dependency-light. Ask the user before adding the crate.
5. Wire three triggers: GANI frame `sound` when a frame is first shown, the GS1 `play` family in `offline/mod.rs` and `scripting`, and a short fixed list of built-in events (sword swing, bomb, item pickup, chest).
6. Missing file: call the existing `send_wantfile`, then retry on the next trigger.
7. Music: play only formats you can decode (WAV, later OGG/MP3). Do not try to emulate MIDI; log "music skipped: midi" and carry on. This is a design call for the user.

How to test

- Unit tests with a recording `SoundOut` mock: feed a gani with `PLAYSOUND`, check calls, the 8-voice cap, pan sign and the loop prefix.
- Lab: the lab gserver world has a `sounds` folder; log in with Seihai, walk, and request a sound file with WANTFILE to see the `PLO_FILE` path (private display only).

NEEDS

nothing for `.wav` effects. (MIDI/MP3 backend choice is a design decision, not missing research. The exact 2.22 extension-to-backend dispatch is only needed if you want BASS parity.)

`client-internals/sound-and-music.md`

Sound and music: 2.05 rev 2, compared with 2.17 / 2.22

Date: 2026-10-21

The 2.05 part is derived from the 2.05 rev 2 binary (sound path, options reader, GS1 sound commands). The later-client part comes from imports, option keys, and strings of `Graal_2171.exe`, `Graal_222.exe`, and `Graal_231.exe`. Names only, no code. Tags: **[2.05]** · **[2.17]** · **[2.22]** · **[inf]**.

2.05 at a glance

What	Backend
Sound effects (`.wav`)	DelphiX `TDXSound` / `TDXWaveList` on DirectSound
Music, Windows MCI path	invisible VCL `TMediaPlayer` (MCI, `mciSendCommand`)
Music, Winamp path	`TWinamp` component that remote-controls a running Winamp 1.x/2.x
Music volume	`midiOutSetVolume` (MCI path) or Winamp IPC messages

Options (`graalconfig.txt`)

Key	Meaning
`musicon`	play music at all
`usewinamp`	send music to Winamp instead of MCI
`soundfxon`	play `.wav` effects
`reversestereo`	flip the pan sign

The backend is chosen when the game starts and again whenever the More Options dialog closes. Switching stops the current track, frees the old backend, and creates the new one. Turning music off stops the current track. **[2.05]**

One entry point for every sound

All sound requests go through one routine that takes `(file, level-offset x, level-offset y, x, y)`. The GS1 commands, GANI `PLAYSOUND`, item pickups, bombs, swords, and baddies all call it. About two dozen call sites use it. **[2.05]**

1. **Loop prefix.** A leading `+` means "looped". The `+` is stripped. Looped effects are added to the player's looped-sound list so that they can be stopped later.

2. **Missing file.** If the file is not on disk and the client is online, it sends a file request to the server (the same path that fetches images). The sound plays on a later trigger once the file has arrived. **[inf]**
3. **Position.** The listener is the player at (x + 1.5, y + 2). A sound from a neighbouring gmap level is shifted by ±64 tiles first.
    - Pan = (sound x − listener x) × 150, clamped to ±10000 (DirectSound pan units). `reversestereo` negates it.
    - Volume = a negative attenuation that grows with the distance on both axes, clamped at −10000 (DirectSound hundredths of a dB).
4. **Dispatch by extension.**
    - **`.wav`** → effect path (below), only when `soundfxon`.
    - **anything else** → music path (below), only when `musicon`. `.mid` is the common case.

Effect path (`.wav`)

- Fifteen effects are known by name and flagged on use: `arrow, axe, bomb, crush, extra, fire, horse2, item, lift, lift2, put, steps, sword, thunder, water` (`.wav`).
- At most **8** effects play at once. A ninth request is dropped.
- Each file is loaded once into the wave list and cached by name. Later plays reuse the buffer with new pan, volume, and loop settings.
- `stopsound file` finds the cached wave by file name (the path is ignored) and stops it.
- Leaving a level or entering the editor stops every looped effect in the player's list.

Music path: MCI

- An invisible `TMediaPlayer` is created on the main form with automatic device type, so `.mid` goes to the MCI sequencer. Other types depend on the MCI drivers installed on the machine. **[inf]**
- If the requested file differs from the current one, or nothing is playing, the client sets the file name, opens it, and plays.
- For `.mid` the player's completion notification is turned on. A handler on the main form replays the track when it ends, so **MIDI music loops forever** until `stopmidi` or another track.
- `setmusicvolume left,right` (0..1 each) calls `midiOutSetVolume` with left and right scaled to 0..65535. That is the system MIDI-out volume, not a per-track volume.

Music path: Winamp

`TWinamp` is a third-party Delphi component (unit `Winamp`). It controls an external Winamp through the public Winamp window-message API. **[2.05]**

- It finds the player by window class `Winamp v1.x`. If no Winamp is running, it reads the Winamp path from the registry key `Software\CLASSES\Winamp.File\shell\Play\command` and starts it.
- A timer polls the window and status, and fires events on play, stop, pause, title change, and exit.
- Playing a track: if Winamp's current file differs or Winamp is stopped, the client loads the file and sends *play* (IPC_STARTPLAY / the play command).
- `stopmidi` sends *stop* when Winamp is playing or paused.
- `setmusicvolume left,right` maps to Winamp volume `(left + right) / 2 × 255` (IPC_SETVOLUME). The balance maps to Winamp panning.
- The component can also list the formats Winamp's input plugins accept (`winampGetInModule2`). In Winamp mode, music is effectively "whatever Winamp can play", MP3 included.

GS1 sound commands in 2.05

Command	Effect
`play file`	One-shot at the NPC's position (NPC x/y + half its image size).
`playlooped file`	Same, with the + loop prefix.
`stopsound file`	Stop a cached effect by file name.
`stopmidi`	Stop the current music (MCI or Winamp).
`setmusicvolume l,r`	MIDI-out volume (MCI) or Winamp volume and pan.

2.17 / 2.22 / 2.31: BASS replaces DirectSound and Winamp

All three later clients import **`bass.dll`** with the same set of functions: init/start/free/update, sample load/play/stop/free, tracker music load/play/free, file streams, **URL streams**, stream length/position, channel attributes, and the **recording** API. `winmm` is still imported, now with `midiOutGetNumDevs` / `midiOutGetDevCapsA` added. **[2.17][2.22][2.31]**

Area	2.05	2.22
Effects	DelphiX DirectSound wave list, 8-voice cap	BASS samples **[inf]**
Music files	MCI (MIDI) or Winamp (anything Winamp plays)	BASS streams and tracker modules. MIDI still via MCI **[inf]**
Internet radio	none	BASS URL streams, `radio` / `radiovolume` **[inf from names]**
Winamp	`usewinamp`, `TWinamp`	**removed**: no Winamp component, key, or window class
Volume options	none (only `setmusicvolume`)	`midivolume`, `mp3volume`, `radiovolume`, `sfxvolume`, `voicevolume`
Script access	`play`, `playlooped`, `stopsound`, `stopmidi`, `setmusicvolume`	same commands plus `musicpos` / `musiclen` (stream position and length)
Voice chat	none	microphone capture through BASS recording, `hvdi.dll` codec

Voice chat (2.22)

New options: `microon`, `microinputdevice`, `microactivatebyvolume`, `microactivationlevel`, `microactivatekey`, `voicecodec`, `voicevolume`. Status strings: "Could not initialize Microphone", "Microphone enabled", "Green when microphone activated". An `emoticonmicro.png` icon is shown over a talking player.

`hvdi.dll` is the HawkVoice voice library (exports `hvdiPacketEncode` / `Decode`, `VOX`, `AGC`, codec selection). The client captures with BASS, encodes with HawkVoice, and sends the voice packets over the network. **[2.22][inf for the transport]**

`basswma.dll`

The 2.2.2 folder ships `basswma.dll` (the BASS WMA add-on), but its use depends on the version:

- **2.17.1** loads it at run time by name and uses `BASS_WMA_StreamCreateFile` / `StreamPlay` / `StreamFree`. **[2.17]**
- **2.22**, **2.31**, and GraalEditor 2.3.1.1 neither import nor name it. In those builds the DLL is a leftover and WMA music is not supported. **[2.22]**

Which action plays which sound

See `live-checks-editor-offline-sound.md` for a table of engine sounds by action, partly confirmed in a running 2.05 client.

Open

- 2.22 extension-to-backend dispatch (which types go to samples, modules, or streams) needs the 2.22 sound routine traced.
- Exactly what 2.05 does after a missing sound file arrives from the server.

`client-internals/bare-level-names.md`

Bare level names and the `.zelda` extension

Which client tries `.zelda` for a level name with no extension, and whether a missing `.zelda` falls back to the `.graal` of the same stem. The answer is that none does, in any client read.

Read in 26 Windows client builds: 1st Graal to 15th Graal (with the 4th, 5th and 9th alphas), 1.1, 1.1.9, 1.21, 1.28 beta 2, 1.3 beta, 1.31, 1.32, and 2.05.

Findings

- **No client adds `.zelda`.** No build carries `.zelda` as a bare extension string. Its only `.zelda` strings are full built-in level names, such as `level1.zelda`, `cave2.zelda`, `mod26.zelda` and `house12.zelda`. Those are compared against the current level's name or used as fixed warp targets, and are present in 1st Graal to 9th Graal.
- **The one extension rule adds `.graal`, and only in the editor.** Every build has the same rule in the level editor's open and save buttons, and in 1.3 and later also the snapshot save: when the chosen file name has no extension, `.graal` is added.
- **No fallback.** No build has code that swaps one extension for the other. A link or warp to `name.zelda` loads `name.zelda` as written; a missing file is just missing.
- **Contents still decide the format.** The loader decides the format from the file's magic, not its extension (`decomp-findings-2026-10-25-j/03-extensions-and-magic.md`), so a `.graal` name holding a `Z3-V1` level still loads.

For a reimplementation

- Keep a level name exactly as the link, warp or server wrote it.
- Add `.graal` only when the editor saves a name typed without one.
- Pick the reader from the magic.

Confirmed, inferred, not tested

- Confirmed from the programs: there is no bare `.zelda` string, the editor adds `.graal`, and no extension-swap code exists.
- Inferred: that the level loader uses the name unchanged. That follows from there being no other extension string near the load path; the loader's file-open call was not traced to its end.
- Not tested: the Java Zelda Online client, and builds between 1.32 and 2.05 other than those listed.

`client-internals/feature-specs/05-save-load.md`

05 Player save and load

Snapshots read: Graal_analysis `8da6800`, Seihai `06aacfd`, GServer-v2 `TAccount.cpp`, `TPlayer.cpp` (`cleanup`), `TPlayerLogin.cpp`.

What it is

For every client from 2.00 on, **the server owns the save**. The 2.05 client contains the string "Save games are not supported anymore", and the offline F2/F3 save games ended with 1.41. The client has nothing to save except its local files (`graalconfig.txt`, `ignoredaccounts.txt`, `buddyaccounts.txt`, per-account profile file, web-level cache `GWEBL001`).

What a client must do is **consume the login burst correctly**, because that burst is the loaded save.

Server file (GRACC001, `accounts/<name>.txt`) [srv]

Text file, first line `GRACC001`, then `KEY value` lines, CRLF on write. Keys read by `TAccount::loadAccount`: NAME, NICK (max 223 chars), COMMUNITYNAME, LEVEL, X, Y, Z, MAXHP, HP, RUPEES, ANI, ARROWS, BOMBS, GLOVEP, SHIELDP, SWORDP, BOWP, BOW, HEAD, BODY, SWORD, SHIELD, COLORS (5 comma-separated), SPRITE, STATUS, MP, AP, APCOUNTER, ONSECS, IP, LANGUAGE, KILLS, DEATHS, RATING, DEVIATION, LASTSPARTIME, FLAG (`name=value`, repeated), ATTR1..ATTR30, WEAPON (repeated), CHEST (repeated: `x:y:level` style string), then the admin block: BANNED, BANREASON, BANLENGTH, COMMENTS, EMAIL, LOCALRIGHTS, IPRANGE, LOADONLY, FOLDERRIGHT (repeated), LASTFOLDER. - Unknown account: the server clones `accounts/defaultaccount.txt`, applies `startlevel`/`startx`/`starty` from `serveroptions` if set, and saves the new file. - `guest` accounts are load-only and get a random `pc:` community name. - Saved on disconnect (`TPlayer::cleanup`, only clients that are not load-only) and every 300 s while connected (`doTimedEvents`). Not saved if the listserver login failed.

How the save reaches the client (login burst, `sendLogin`) [srv][cap]

Order (`TPlayer::sendLogin` then `sendLoginClient`): signature, own props (`PLO_PLAYERPROPS` 9 with the loaded values; no level or position props), `PLO_CLEARWEAPONS`, one `PLO_FLAGSET` (28) per saved player flag, the server flags, `NPCWEAPONDEL` for "Bomb" and "Bow" (the client adds them itself), the account's weapons (`NPCWEAPONADD` 33 and `DEFAULTWEAPON` 43), protected weapons and classes, then the warp: `PLO_PLAYERWARP` (14) to the saved level and x/y followed by the level data. After the level come bigmap/minimap/RPG window (2.1+), `PLO_STARTMESSAGE` (41) and an empty `PLO_SERVERTEXT` (82). Chest open state arrives inside each level's chest packets (03). The saved `LEVEL`, `X`, `Y` therefore arrive as the first warp, not as props. Level and position are stored, so reconnecting resumes where the player left.

Seihai today

- Consumes the burst: props (partly), flags (`SessionState.flags`), weapons, default weapons, warp, board, chests list from LEVELCHEST (`opened_chests`).
- No local save. Offline mode in web-Seihai design notes has its own JSON save (`SaveV1` under `formats/seihai-systems.md`), not related to GRACC001.
- Seihai writes only its own `settings.rs` file.

Steps for Claude

1. Nothing to implement for online play beyond keeping the full self prop stream (03) and flags (done).
2. Keep `login_state` in the session: level, x, y from the first `PLAYERWARP`; log it in `session_probe` output (names only).
3. Optional offline save: keep the existing offline model; do not invent a GRACC001 writer in the client. If a local server is wanted, use GServer-v2 in the lab instead.
4. Add a lab test that logs in twice and asserts the second `PLAYERWARP` equals the position saved at the first logout.

How to test

- Lab: log in with the placeholder account, move, send a chat, disconnect, then read `servers/lab/accounts/<name>.txt` (X, Y, LEVEL, HP, RUPEES). Reconnect and confirm the warp matches. Never print credentials.
- Or replay a captured login burst through `SessionState`.

NEEDS

nothing.

`client-internals/config-and-input.md`

Config and input

Date: 2026-10-14

Derived from observed client behavior, the public config key list, and format notes. No decompile material.

graalconfig.txt

> **Superseded** by `graalconfig-reference.md`, which lists every key 2.05 reads and writes. `serveraddr`, `loadcolorsfromserver` and `gamemode` below are not read by 2.05. The "password mask literal" is the fixed DES key for saved passwords (`saved-passwords.md`).

The client reads approximately 27 keys from `graalconfig.txt`. Known keys fall into four groups:

Connection - `serveraddr` — host override - `fixedport` — TCP port override - `fixedudpport` — UDP port override - `dontconnectlevels` — skip automatic level connect - `dontloadlistheads` — skip list header images - `loadcolorsfromserver` — pull color table from server

UI - `sortplayersbyname` - `limitnicknames` / `nicknamelimit` - `gamemode` - `language` - `font` / `fontsize`

Audio and input - `musicon` - `joystickon` - `nodirectinput` - `noudp` - `reversestereo`

Compatibility - `nodecompressionerror` - `nohtmlimages` / `nohtmlinpms` - `nomassmessages` / `notoalls` - `saveimages` - `compabilitydraw` — toggles a DirectDraw compatibility path

A password mask literal exists in the binary and is not decrypted. It is used only for display.

Device path

Joystick support is present. The device is activated when a specific internal bit is set. The WinMM joystick identifier is stored separately from the main window object. An offline session that does not need joystick input can ignore this path.

Drop files

Dragging a file onto the client window is handled only when an internal flag is set (the same flag that enables the editor drop target).

- A single `.txt` extension spawns a script NPC.
- Three specific image extensions (`.gif`, `.png`, `.mng`) spawn an image NPC.
- Anything else is treated as a level load.

This matches the offline observation that local level files, GANIs, and NPCs can be loaded from a `levels/` tree. An offline session that already accepts a dropped `.nw` or `.graal` file is exercising the same path.

Offline implication

None of the config keys are required for the offline slice. The drop-files behavior is the only part that intersects with current offline work: a dropped level file should load, and a dropped image or script should spawn the corresponding NPC. That is already the expected next step after collision and CmdCall.

Update 2026-10-21: 2.05 vs 2.22 graalconfig

Compared the 2.22 `graalconfig.txt` (user's Drive, 86 keys) with the 2.05 string table. **[2.05][doc]**

- **Present in 2.05:** `compabilitydraw`, `dontconnectlevels`, `dontloadlistheads`, `dontsavepasswords`, `dontsavepms`, `dontusedirectx7`, `fixedport`, `fixedudpport`, `font`, `fontsize`, `fullscreenmode`, `head`, `joyselected`, `joystickon`, `language`, `limitnicknames`, `nicknamelimit`, `nodecompressionerror`, `nodirectinput`, `nohtmlimages`, `nohtmlinpms`, `nomassmessages`, `notoalls`, `noudp`, `reversestereo`, `saveimages`, `sortplayersbyname`, `waitvertical`, `musicon`. That is about 29 keys, which fits the "~27 keys" estimate above.
- **Key bindings:** the only related 2.05 string is the bare prefix `keyset_`, so 2.05 probably builds `keyset_<action>_key0..2` and `keyset_<action>_joystickbutton` names at runtime [inf]. The actions in 2.22 are up, left, down, right, item, attack, action, map, switchtochat, itemlist, and pause, with defaults arrows/IJKL, D, S/space, A, M, Tab, Q, and P.
- **2.22-only** (absent in 2.05): `automapping`, the `micro*`/`voice*` voice-chat keys, `midivolume`/`mp3volume`/`radiovolume`/`sfxvolume`, `startinfullscreen`, `quickplayerworlds`.
- `fixedudpport` defaults to 14899 and `noudp=true` in 2.22. In 2.05 the UDP channel is a peer-to-peer channel between players; see `protocol/udp-p2p.md`.

`client-internals/graalconfig-reference.md`

graalconfig.txt and registry reference (client 2.05)

Date: 2026-10-21

Traced from the 2.05 rev 2 options reader and writer, the More Options and Player List Options dialogs, and their form resources. Checked against a real `graalconfig.txt` written by the 2.22 client, which uses the same keys in the same order. Tags: **[2.05]** read from the client · **[2.22]** seen in a 2.22 file · **[inf]** inferred.

Where settings live

Store	Location	Holds
`graalconfig.txt`	Next to the exe	Every option on the More Options and Player List Options dialogs, the head, language, font, joystick and key bindings
Registry	`HKEY_CURRENT_USER\Software\Graal\Client`	`nickname`, `accountname`, `password`, and one `<account>:pass` value per account

The file is a plain `key=value` list, read with Delphi `TStringList.Values`. Key lookup is case-insensitive, and order doesn't matter on read. **[2.05]**

When it is read and written

- **Read** once, when the options form is created at startup. **[2.05]**
- **Written** in full every time the More Options or Player List Options dialog closes. The writer always emits every key in the fixed order below, so hand edits that add other keys are lost on the next save. **[2.05]**
- **If the file is missing**, the client keeps its built-in defaults and sets `language` to the English name of the Windows user locale (for example `English` or `German`). **[2.05]**

Value parsing

Kind	Rule	Pitfall
Boolean	`true` when the value, lowercased, is exactly `true`. Anything else is `false`. Written as `true` / `false`.	A **missing** boolean key reads as `false`, not as the default. A partial file therefore switches off the options that default to on: `musicon`, `soundfxon`, `saveimages`, `nohtmlimages`, `joystickon`.
Integer	`StrToInt`. A conversion error is caught and the default is kept.	An empty or missing integer key keeps the default.
String	Used only when non-empty.	

Keys, in file order

UI labels are the dialog captions. MO = More Options, PL = Player List Options.

Key	Type	Default	UI	Effect
`waitvertical`	bool	false	MO "Wait for vertical blank (slower but smoother)"	Vsync on flip
`musicon`	bool	true	MO "Music support"	Turning it off stops the music player **[inf]**
`usewinamp`	bool	false	MO "Use WinAmp for music"	Music goes through Winamp

Key	Type	Default	UI	Effect
`soundfxon`	bool	true	MO "Sound FX's enabled"	
`reversestereo`	bool	false	MO "Reverse stereo channels"	
`saveimages`	bool	true	MO "Save downloaded images (to)"	Turning it on writes the images already in memory to disk **[inf]**
`dontsavepms`	bool	false	PL "Don't save PMs/ Profiles"	
`compabilitydraw`	bool	false	MO "Draw in compatibility mode (very slow)"	Spelling as in the client
`nodecompressionerror`	bool	false	MO "Don't show decompression errors"	
`head1`, then `head`	string	`Fung Si Yun`	none	Read in that order, so `head` wins. Only `head=` is written. The default value is a literal in the client. **[2.05]** The 2.22 file still holds it. **[2.22]**
`joystickon`	bool	true	MO "Joystick support"	
`nodirectinput`	bool	false	MO "Don't use DirectInput"	
`joyselected` (also reads `joyselected1`)	string	`(none)`	Controls frame	`(none)`, `Joystick1` or `Joystick2`
`keyset_<action>_joystickbutton`	string	per action	Controls frame	One of `Joy_Up`, `Joy_Down`, `Joy_Left`, `Joy_Right`, `Button1`… `Button12`
`keyset_<action>_key0` … `_key2`	string	per action	Controls frame	Three key slots per action, by key name (`Up`, `Num 8`, `Space`, `Tab`, …). Empty means unbound.
`language`	string	`English`	Connect frame language box	
`font`	string	`Tempus Sans ITC`	MO "Font:"	Chat and name font. The 2.22 folder ships this font file. Defaults per build in `chat-text.md`
`fontsize`	int	24	MO "Size:"	
`fixedport`	bool	false	MO "Use fixed port for data packages:"	
`fixedudpport`	int	14899	MO port box	Used when `fixedport` is true

Key	Type	Default	UI	Effect
`noudp`	bool	false	MO "Don't use UDP for data packages"	Changing the UDP options restarts the UDP socket **[inf]**
`nomassmessages`	bool	false	PL "Ignore mass messages"	
`nohtmlinpms`	bool	false	PL "Don't use HTML for PMs"	
`limitnicknames`	bool	false	MO "Limit the lenght of nicknames to"	
`nicknamelimit`	int	50	MO length box	
`dontconnectlevels`	bool	false	MO "Don't connect levels"	Turning it on drops the cached neighbour levels **[inf]**
`notoalls`	bool	false	MO "Ignore toalls"	Turning it on also clears the toall history **[inf]**
`nohtmlimages`	bool	true	PL "No images in PMs"	
`sortplayersbyname`	bool	false	PL "Playerlist: sort by nickname"	
`dontloadlistheads`	bool	false	MO "Don't load heads for the playerlist / map"	
`dontsavepasswords`	bool	false	MO "Don't save the passwords"	See below
`dontusedirectx7`	bool	false	MO "Don't use DirectX7"	
`fullscreenmode`	string	`1024x768`	MO "Fullscreen mode:"	One of `640x480`, `800x600`, `1024x768`, `1280x1024`, `1600x1200`. Any other value keeps the default resolution.

The joystick and key block is written right after `head=`. The 11 actions, in file order, are `up`, `left`, `down`, `right`, `item`, `attack`, `action`, `map`, `switchtochat`, `itemlist`, `pause`. **[2.05]**

Key binding rules on load: **[2.05]** - A key name that is already bound earlier in the file is treated as `-` (unbound), so one key can't drive two slots. - Binding a key that another slot already holds moves it, and the other slot gets the old key.

Default joystick buttons, in action order: `Joy_Up`, `Joy_Left`, `Joy_Down`, `Joy_Right`, `Button1`…`Button5`, `Button9`, `Button10`. **[2.05]** The 2.22 file matches. **[2.22]**

Keys in older notes that 2.05 does not read

`serveraddr`, `loadcolorsfromserver` and `gamemode` (listed in `config-and-input.md`) are not read or written by 2.05. If they exist, they belong to later clients. **[2.05]**

Registry values

Value under `HKCU\Software\Graal\Client`	Read	Written
`nickname`	At startup	When the dialogs close
`accountname`	At startup. It is a comma list, and the first entry is the current account.	When the dialogs close
`password`	At startup, decrypted	When the dialogs close, unless `dontsavepasswords` is true
`<account>:pass`	At startup for the current account, and again when the account box changes	Same as `password`

When the More Options dialog closes with "Don't save the passwords" ticked, the client deletes `password` and every value whose name ends in `:pass` from that key. **[2.05]**

Password values are encrypted with a fixed key. See `saved-passwords.md`.

`client-internals/saved-passwords.md`

Saved passwords and the client's DES string cipher

Date: 2026-10-21

Traced from client 2.05 rev 2. Tags: **[2.05]** read from the client · **[test]** checked with `tools/file-formats/graaldes.py`.

Warning for anyone holding old installs

The client stores account passwords in the registry under `HKEY_CURRENT_USER\Software\Graal\Client` (`password`, and `<account>:pass` per account). They are DES-encrypted with an 8-byte key that is the same in every copy of the client: `~2%$zsd^`. **[2.05]**

The same key and string form are already used by 1.3 test (Nov 1999): a value saved by 1.31/1.32 in a lab run decrypts with `graaldes.py`, and the key string is in every client from 1.3 test to 1.39 (`live-trace-early.md`). So the warning covers those old installs too.

That means anyone who can read the registry, or a `.reg` export or backup of that key, can recover the password with no secret at all. Treat these values as plain text. If an old machine, disk image or exported profile is shared, delete the values first, or tick "Don't save the passwords" in More Options (the client then deletes them on close).

Cipher

- DES in ECB mode, standard tables, with one twist: **each key byte is bit-reversed** before the key schedule (bit 0 becomes bit 7). This is how the Acme Labs Java `DesCipher` handles keys, and the client's DES unit behaves the same. **[2.05] [test]**
- Equivalent: standard DES with the key `bitreverse(k[0]) … bitreverse(k[7])`.

String format (48 characters)

Encrypt: 1. Truncate the plaintext to 23 bytes. 2. Build 24 bytes: `len`, the plaintext, then random padding bytes. 3. Encrypt as three DES blocks. 4. Write each of the 24 bytes as two characters: `(b >> 4) + 0x20`, `(b & 0x0F) + 0x20`.

Decrypt reverses this. Input shorter than 48 characters gives an empty string. The first decrypted byte is the length, and values of 0 or ≥ 24 give an empty string. **[2.05]**

Because the padding is random, the same password encrypts to a different string each time.

Other uses of the same cipher

- The hidden listserver host: the server list frame builds a different 8-byte key and a 48-character ciphertext one character at a time. It decrypts to `<list-host-classic>`, port `14921`. **[2.05] [test]**
- The web-level cache uses the same DES routine with per-block keys. See `formats/gwebl001-cache.md`.

Tool

`tools/file-formats/graaldes.py` (pure Python):

```
python3 graaldes.py '<48-char value>'
```

It implements `GraalDES` (block cipher), `string_decrypt` and `string_encrypt`. It reproduces the listserver string and round-trips its own output.

`client-internals/des-and-hidden-strings.md`

Hidden strings and 2.22 saved passwords (2.05, 2.22)

Date: 2026-10-21

Tags: **[2.05]** read from the 2.05 rev 2 binary · **[2.22]** read from the 2.22 binary · **[cap]** checked in the lab (`tools/lab-setup.md`) · **[inf]** inference.

DES, saved passwords and the web cache (pointers)

The 2.05 DES routine (standard DES-ECB with each key byte bit-reversed), the 48-character string form and the 2.05 saved-password scheme are written up in `client-internals/saved-passwords.md`, and the web-level cache cipher in `formats/gwebl001-cache.md` (both added by PR #6). This note only adds the parts those files don't cover.

2.22 saved passwords [2.22] - Same 48-character scheme as 2.05, with one change. A saved value of 49 characters starting with a marker character is decrypted with a second key held by the options form. Any other value is decrypted with the first key, so old saved passwords still load. - New saves use the second key and the marker. - The account name is also stored hidden, using the byte transform below.

Hidden strings

2.05 listserver host [2.05][cap] - Uses the DES string form from `saved-passwords.md`. The server-list form builds an 8-byte DES key (`2B 77 77 A7 B4 31 21 41`) and a 48-character ciphertext one `Chr()` at a time, so neither appears in the string table. - It decrypts to `<list-host-classic>`. The port (`14921`) and the first packet text (`newmain`) are built the same way, as plain characters.

2.22 transform [2.22][cap] - `dec(b) = (NOT b − 10) & 0xFF` and `enc(b) = NOT (b + 10)`. - Used for: - the in-memory copy of the three listserver host names (`<list-host-reborn>`, `.<list-host-graalcc-suffix>`, `<list-host-graalbiz>`) and the port (`14922`) - the account name in the config - the account name before it goes into the game-server login packet - The host names and port themselves are read at start-up from `arrays2.dat`, where they are plain text (checked against `arrays2.sig`). Only the copy held in memory is transformed, so a memory search for the plain names finds nothing. See `protocol/listserver.md`.

2.22 version token [2.22] - The game-server login builds `GNW03014` one character at a time. - On the wire the login line is: `GCHAR 5` (client type 5, gen 5), `GCHAR key`, the token, `GCHAR len` + account, `GCHAR len` + password. **[cap]** - It is followed at once by id 37 (language) and id 39, a 10-byte value that is the same at every login of a given build (raw bytes, not two GINT5 words). What it is computed from is not traced. **[cap]**

Notes for tool writers

- To redirect 2.05, a hosts entry for `<list-host-classic>` is enough.
- To redirect 2.22, hosts entries for the three `listserver.*` names are enough. It tries them in order. Editing `arrays2.dat` instead would have to get past the `arrays2.sig` check, which the lab did not test.

- None of these schemes protect anything. They only keep the strings out of a casual `strings` dump.

`client-internals/feature-specs/08-tampercheck.md`

08 Tampercheck (PLI 39) for 2.17+ and the process list

Snapshots read: Graal_analysis `8da6800` (`protocol/client-versions.md`, `version-profiles/2.22-2.31.md/.toml`), Seihai `06aacfd` (`protocol/profile.rs`), GServer-v2 (`TPlayer.cpp` `msgPLI_MAPINFO`, `msgPLI_PROCESSLIST`, `TPlayerLogin.cpp`), the public-server report. Folded here: **PLO_LISTPROCESSES 182** and PLI 44/45 process list.

Naming trap

GServer-v2's older header calls PLI 39 `PLI_MAPINFO` ("might be gmap related"). The Graal_analysis notes call it `PLI_TAMPERCHECK`. **Client to server 39** is not **server to client 39** (`PLO_LEVELMODTIME`, see 06). Never mix them.

What it is [bin][cap]

- New in 2.17; absent in 2.05 and 1.x. Sent right after `PLI_LANGUAGE` (37) at login: it is the second message of the client's first gen-3/gen-5 frame, after `PLI 37 English`.
- Payload: **10 raw bytes, not two GINT5 words** (corrected 2026-10-24 from real-client lab captures). Bytes below 0x20 occur in the 2.17 and 2.22 values, which a GINT5 byte can never be, and no value contains 0x0a, so it needs no escaping on the line-framed wire.
- It is a **per-build constant**: the same 10 bytes at every login, for different accounts and different login keys.

Build	Token	10 bytes (hex)	Re-sent
2.17	`GNW22122`	`2a7bad195e231243d82c`	no (once at login; nothing more in about 90 s)
2.22	`GNW03014`	`250e4d0f962b03ede631`	once per gmap edge crossing; no periodic send in about 100 s
2.31	`GNW28015`	`291655b7602e87c9eb82`	about every 3.2 s

The same table is in `version-profiles/tampercheck-values.tsv`. The 2.22 value matches the earlier TOML record, so that record is confirmed. - 2.31 only: it also sends `PLI_SHOWIMGPLAYER` (24) with a 1-byte payload (value 9 seen) about every 5.1 s, and `PLI_ENTERLEVEL` (46, 1 byte, value 1) on every level entry (login and each gmap edge crossing; it does not come with a re-sent 39, which 2.31 sends on its 3.2 s timer only). Neither was in the profile notes; the meaning of the id 24 payload is unknown. (Lab batch c: 2.22 also sends 24 = 9 about every 5.1 s while another player is in its level, 2.31 always; 2.31 sends 46 on doors, links and server warps too and re-sends no 39 there. 2.22 sends no 46 but sends 39 in the same frame as each client-initiated level-link request, not after a server-side warp: `../decomp-findings-2026-10-24-b/08-lab-findings-levelchange-24-178.md`.) (Id 46 is settled: see the settled index in `README.md` and `../decomp-findings-2026-10-24-b/07-lab-findings-46-186-200.md`; door and warp entries untested.) - **What the bytes are (decompile pass, 2026-10-24; all six lab values reproduced offline) [bin]:** two 32-bit words A then B, each written as 5 bytes in a form that is **not** GINT5 and **not masked**: byte 1 = `((v >> 28) & 0x0F) + 0x20`, bytes 2–4 = `((v >> 21) + 0x20) & 0xFF`, `((v >> 14) + 0x20) & 0xFF`, `((v >> 7) + 0x20) & 0xFF`, byte 5 = `(v & 0x7F) + 0x20`. Only bytes 1 and 5 are range-limited, which is why bytes below 0x20 occur. Both words are standard reflected CRC-32 (polynomial `0xEDB88320`, start `0xFFFFFFFF`) over the client's **own memory image** (the running exe), chained: the first call ends with the usual final inversion and its result seeds the next call, and the last call of each word skips the final inversion. Word A covers 8–9 contiguous ranges of the code section; word B covers one large code range plus one block of static data (a table of string pointers). Nothing depends on

the login key, account, level or time, so the result is a per-build constant that changes only if the exe bytes change. Words per build: `version-profiles/tampercheck-words.tsv`; full write-up: `client-internals/decomp-findings-2026-10-24-b/01-tamper-check-id39.md`. - **2.31 re-send timing [bin]:** a 50 ms main-form timer keeps a counter modulo 64 (the length of the 64-character alphabet string it uses), so one cycle is 64 × 50 ms = 3.2 s. Phases: 1 anti-debug probe and a 5000 ms timer is armed; 10 recompute word A; 20 scan top-level windows against an obfuscated blacklist (not decoded); 30 recompute word B; 31 send PLI 39. The 5.1 s PLI 24 cadence fits the 5000 ms timer (inference). 2.17.1 has no such timer and 2.22 re-sends only from its level-change routine. See `client-internals/decomp-findings-2026-10-24-b/03-periodic-ids-2.31.md`.

Exact server behaviour [srv]

`TPlayer::msgPLI_MAPINFO` (id 39) reads the rest of the packet and returns. **GServer-v2 does not validate it.** The lab therefore cannot tell you whether a different value would be rejected. Real servers may check; unknown. Process list: the server sends `PLO_LISTPROCESSES` (182) once during login (2.17+). The client can answer `PLI_PROCESSLIST` (44) with a list of running programs (inferred from the names and the server's parse, not captured); `msgPLI_PROCESSLIST` parses it into a vector and ignores it. A later copy of the server also has 45 (process running). Anti-cheat only. Also relevant: `msgPLI_NULL` counts unknown ids and disconnects after more than 5, so never send ids that are not in the client's profile `pli` set.

Public servers

The 2026-10-24 run logged in to 4 public servers with 2.22 and **no id 39**; none disconnected within 20 s. That shows presence is not enforced at login on those four, not that it is never enforced. A real 2.31 client resends it every ~3.2 s, so a periodic check is plausible there, but no server behaviour has been observed.

Seihai today

- Profile flag `tampercheck` exists. `send_post_login` sends LANGUAGE only (no id 39), per the report. In the lab (2026-10-24) the merged Seihai logged in as 2.17, 2.22 and 2.31 without sending it, and GServer-v2 accepted that.
- 182 and 44 are not handled.

Steps for Claude

1. The bytes are a constant per unmodified build, so replay the table values; do not try to recompute them at run time.
2. In the 2.17/2.22/2.31 profiles add `tampercheck_bytes: Option<[u8;10]>` from the table above; send `PLI 39` right after LANGUAGE when the profile has `Some`. Version-specific re-sends (2.22 on each gmap edge crossing, 2.31 about every 3.2 s) only matter if a server is seen to enforce it.
3. Make it opt-in at first (`--send-tampercheck`) so the public-server default stays "same as the 4 servers that accepted us".
4. Handle 182 by sending nothing (or an empty `PLI_PROCESSLIST` if a server complains); log it.
5. Add a test asserting that the message after LANGUAGE in the first post-login frame is id 39 followed by exactly the 10 raw profile bytes.

How to test

- Lab with GServer-v2: confirm the extra packet causes no error and no invalid-packet counter increase (the handler exists). Compare Seihai's frames with the 2.22 capture frame by frame (`examples/lab_replay`).

- Cannot test enforcement locally.

NEEDS

- ~~For enforcement only: the 2.22 id-39 value builder~~ Done 2026-10-24: the decompile shows how the bytes are built (above). Computing them for a patched exe needs that exe's own code ranges, which are inside each client binary; replaying the per-build constant is enough for an unpatched client.
- The decompiles only show the sender. There is no check on the client's receive side, so whether any server enforces the value is still unknown.
- ~~For 2.17 and 2.31 values~~ Done 2026-10-24: captured from the real clients under Wine in the loopback lab (table above).

`client-internals/feature-specs/07-udp-peer.md`

07 UDP peer channel

Snapshots read: Graal_analysis `8da6800` (`protocol/udp-p2p.md`, `packet-layouts.md`, `tools/lab-setup.md`), Seihai `06aacfd` (`net/udp.rs`). Folded here from the public-server report: **PLO_HASNPCSERVER 44**, **PLO_ISLEADER 10** (as it relates to TCP fallback).

What it is

A client-to-client UDP channel in 2.05. The **server never sees it** (GServer-v2 has no implementation); the gserver only relays each player's address (props 30 IPADDR i5 and 31 UDPPORT i3) in `PLO_OTHERPLPROPS`. Layout: datagram = `seq` byte + GCHAR UDP id + fields, all XORed with `recipientId XOR 1`. No zlib, no length prefix. Receiver accepts only from a known peer ip:port, drops duplicate seq. Ids: 0 move, 10/11 bomb add/del, 12/13 arrow/fire spy, 14 explosion, 15 showimg, 16/17 horse, 18 throw carried, 19/20 item add/del, 21 remote hit, 22 chat, 23 level name (receive only), 24 entered-level (x*2*, *y*2, level) to the 3x3 gmap neighbourhood, 25 hit objects, 26 trigger action. Payloads for 0, 15, 21, 23, 24 are in the note.

Routing rule [2.05]

A level action goes UDP-only when every other player in the level has a UDP port; otherwise it goes by UDP to peers that have one **and** by TCP to the server. Player props still go over TCP. **GServer-v2's `PLO_HASNPCSERVER` (44)**: if sent, 2.05 sends the TCP copy too.

Consequence for Seihai: UDP is optional. If Seihai advertises no UDP port (prop 31 = 0 or absent), 2.05 peers send every combat/object message over TCP, and Seihai receives them as normal PLO packets. UDP only saves latency. Seihai's profile has `udp` and `udp_off_by_default` flags already.

Seihai today

- `net/udp.rs`: `encode_udp`, `decode_udp`, `udp_key`, id constants, socket bind helper; lab-verified with 2- and 3-player pcaps (`examples/udp_replay`).
- Not wired into the session loop: no socket is polled, no props 30/31 are sent, UDP id handlers do not exist.
- PLO 44 and 10 are unhandled.

Steps for Claude

1. Keep UDP off by default for all profiles (document that peers fall back to TCP). Do **not** send prop 31 unless the user enables it.
2. Optional path, only when enabled: bind (`fixedudpport` or ephemeral), send props 30/31 once after login, poll the socket every frame. The XOR key is `recipientId XOR 1`: to send to peer P use `udp_key(P)`; on receive, use `udp_key(own player id)`.
3. Map each UDP id to the same internal event as its TCP twin (bomb 10 = PLO 11, etc., see 02) so there is one code path.
4. Send id 0 movement only when the position changed; fields X*2*, *Y*2, SPRITE, STATUS, CARRYSPRITE, animation letter (first letter of the gani name, upper or lower case as seen: `W`, `I`, `S`). Do not send id 23.
5. Store HASNPCSERVER (44) and ISLEADER (10) flags.

How to test

- `examples/udp_replay` against the existing pcaps (private, never committed; they contain the lab password in plaintext).
- Lab: two Wine 2.05 clients plus Seihai with UDP disabled; confirm bombs arrive over TCP at Seihai. Then enable UDP in Seihai and confirm datagrams decode.

NEEDS

nothing to implement. Open questions that do not block: which client version sends UDP id 23, and the meaning of the last byte in id 0 beyond "first letter of the animation" (needs a 2.22/2.31 UDP capture if those versions are to be supported; 2.22 defaults to `noudp=true`).

`client-internals/offline-play.md`

Offline play notes

Date: 2026-10-14

Requirement

Seihai must support playing levels offline (no listserver / gserver connection).

Observations from original client

- Tutorial levels are embedded in the executable (not loaded from disk).
- `2dadventureclient` already demonstrates local player + level simulation using the shared game object model.
- Level files, GANIs, NPCs, and links can be loaded from a local `levels/` tree.
- The original client's editor (F4) and snapshot features operate on local level data.

Current status

- `.nw` parser + board-modify applicator (`Level::apply_board_modify`).
- `OfflineSession` can load a level and apply `BoardModify` updates.
- GANI parser extracts SPRITE defs, ANI frames (placements), and DEFAULT* fields.

Remaining for offline play

1. Tile collision using the parsed boards.
2. NPC script execution (seihaiscript).
3. Rendering the tile data.

See `offline-tutorial-205.md` and `level-editor-205.md` for what the 2.05 client itself does offline.